Construcción de microservicios con JWT, H2, Swagger, OpenFeign, filtro de seguridad y Docker: Laboratorio 04

[Nombre completo del estudiante]

[Programa académico], [Nombre de la universidad]

Ingeniería de Software III

[Nombre del docente]

8 de noviembre de 2026

Construcción de microservicios con JWT, H2, Swagger, OpenFeign, filtro de seguridad y Docker

El presente informe documenta el desarrollo del Laboratorio 04 de la asignatura Ingeniería de Software III. El propósito del laboratorio fue partir de un proyecto base organizado por capas y hacerlo evolucionar hacia una solución de dos microservicios: *auth-service*, encargado de autenticar usuarios y emitir tokens JWT, y *consulta-service*, que expone la consulta de estudiantes protegida mediante esos tokens. Como proyecto base se utilizó el repositorio *demo-academico-application*, publicado en GitHub para el curso (ojugama, s.f.).

La solución integra persistencia embebida en H2, cifrado de contraseñas con BCrypt (Provos y Mazières, 1999), tokens JSON Web Token (Jones et al., 2015), comunicación declarativa entre servicios con OpenFeign, documentación interactiva con Swagger/OpenAPI y despliegue local con Docker Compose. Además de describir la implementación, el informe presenta la evidencia de las pruebas obligatorias realizadas desde Swagger, responde las actividades sobre construcción de imágenes, volúmenes y redes en Docker y describe los problemas de entorno que se resolvieron durante el proceso.

Objetivos

Objetivo general

Construir una solución funcional de dos microservicios en la que uno autentica usuarios y genera tokens JWT y el otro protege la consulta de estudiantes validando dichos tokens, desplegada de forma integrada con Docker Compose.

Objetivos específicos

1. Reutilizar el proyecto base con arquitectura controller, handler, service, repository y model como punto de partida de *consulta-service*.

2. Implementar *auth-service* con usuarios almacenados en H2, contraseñas cifradas con BCrypt y emisión y validación de JWT.
3. Integrar ambos servicios mediante OpenFeign y un filtro de seguridad que intercepte las peticiones a `/api/estudiantes`.
4. Documentar la seguridad en Swagger con el esquema *Bearer* y el botón *Authorize*.
5. Contenerizar ambos servicios y verificar el flujo completo con las pruebas obligatorias.

Marco conceptual

Arquitectura por capas y microservicios

La arquitectura por capas separa la recepción de peticiones (controller), la transformación entre DTO y entidades (handler), las reglas de negocio (service), el acceso a datos (repository) y el modelo persistente (model). Esta separación facilita mantener y probar cada parte por separado y es la base sobre la que luego se dividen las responsabilidades en servicios independientes. Un microservicio es una aplicación pequeña, desplegable de forma independiente y organizada alrededor de una capacidad del negocio, que se comunica con otros servicios mediante mecanismos ligeros como HTTP (Fowler y Lewis, 2014; Newman, 2021).

Autenticación con BCrypt y JWT

BCrypt es una función de derivación de claves diseñada para el almacenamiento de contraseñas. Incorpora una sal aleatoria y un factor de costo ajustable, de modo que el mismo texto produce resúmenes distintos y los ataques por fuerza bruta resultan costosos (Provos y Mazières, 1999). Por esta razón la base de datos de *auth-service* nunca guarda la contraseña en claro, sino el resumen con prefijo `$2a$10$`.

Un JSON Web Token (JWT) es una cadena compacta formada por encabezado, carga útil y firma, que permite transmitir afirmaciones (claims) verificables entre dos partes (Jones et al.,

2015). En este laboratorio el token se firma con HMAC-SHA256 e incluye el usuario (`sub`), el rol, la fecha de emisión y una expiración de una hora. Así el cliente no necesita reenviar usuario y contraseña en cada petición.

OpenFeign, Swagger y Docker

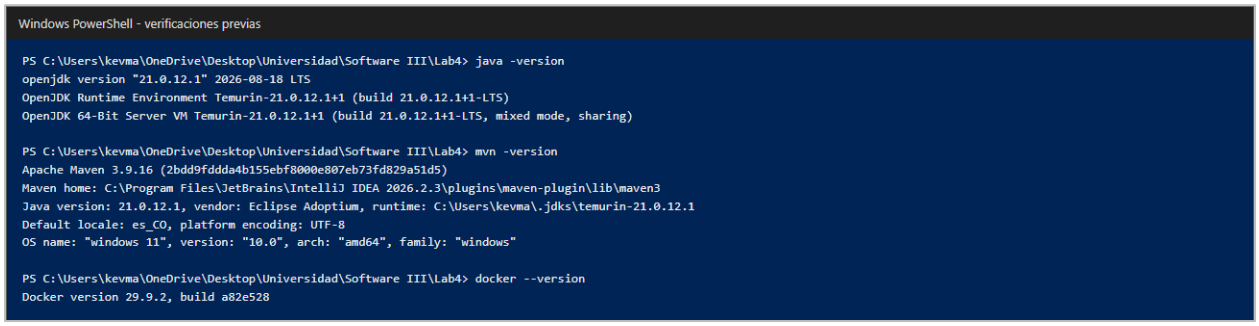
Spring Cloud OpenFeign permite declarar un cliente HTTP como una interfaz Java anotada, lo que hace más legible la comunicación entre servicios que una llamada armada manualmente (Spring, s.f.-b). Swagger UI, construido sobre la especificación OpenAPI (OpenAPI Initiative, 2020), documenta los endpoints y permite probarlos desde el navegador, incluido el envío del encabezado `Authorization` mediante el botón *Authorize*. Finalmente, Docker Compose define y ejecuta aplicaciones de varios contenedores a partir de un único archivo YAML, lo que hace repetible el despliegue local (Docker, s.f.).

Entorno de trabajo

El laboratorio se desarrolló en Windows 11 con Java 21 (Eclipse Temurin 21.0.12.1), Apache Maven 3.9.16, Spring Boot 3.2.5, Spring Cloud 2023.0.1 y Docker Desktop 29.9.2 con Docker Compose v5.6.1. El JDK y Maven corresponden a las versiones que instala IntelliJ IDEA 2026.2.3, por lo que se usaron directamente desde esa instalación. La Figura 1 muestra la verificación previa exigida en la guía.

Figura 1

Verificación de Java, Maven y Docker



Nota. Salida de los comandos `java -version`, `mvn -version` y `docker --version` ejecutados en PowerShell.

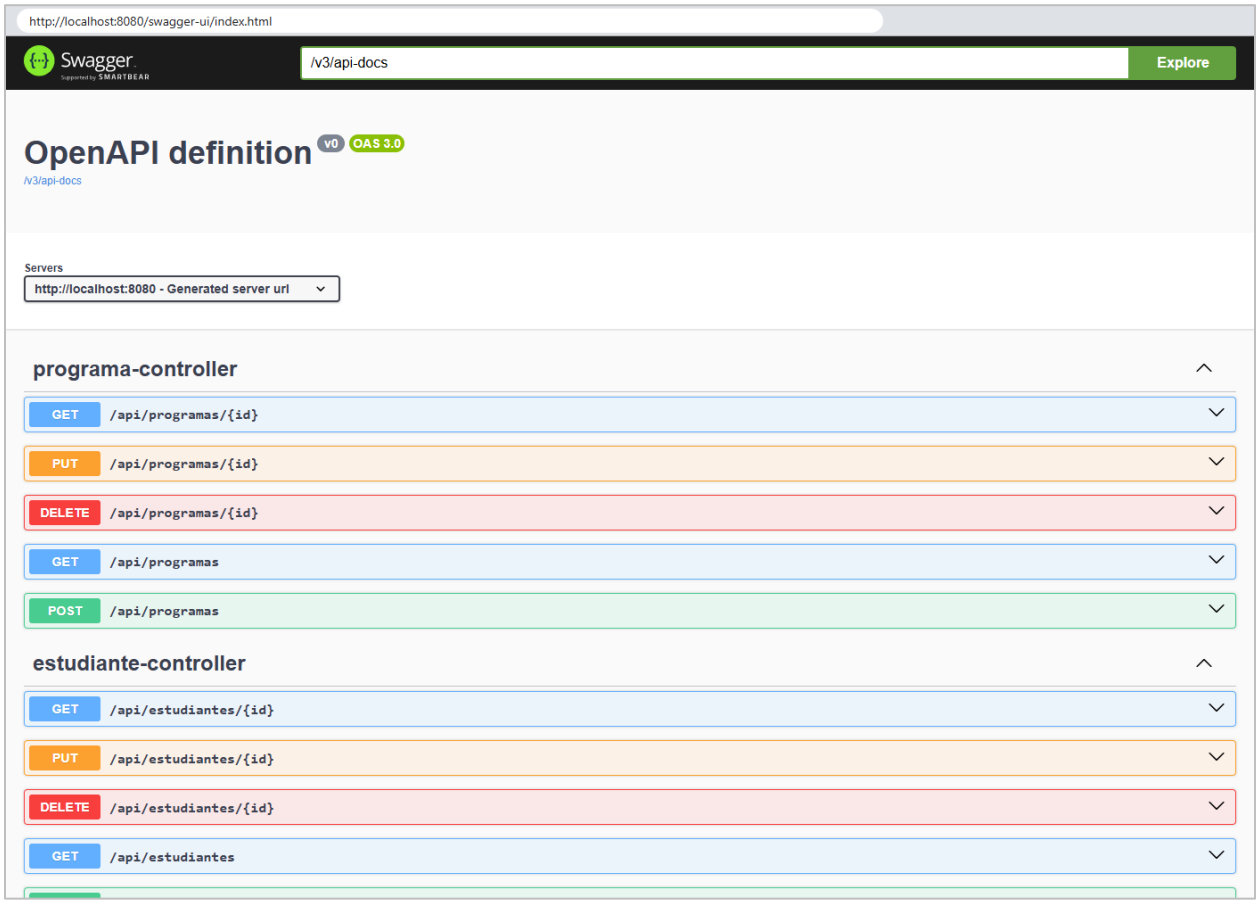
Desarrollo

Parte 0: proyecto base

El repositorio indicado por el docente se descargó en la carpeta `demo-academico`. A diferencia del ejemplo mínimo de la guía, el repositorio ya trae un dominio más amplio organizado por funcionalidad: estudiantes, programas, asignaturas, grupos y matrículas. Cada módulo conserva la misma arquitectura (controller, handler, service, repository, entity y DTO) y comparte los paquetes `common.api` (`ApiResponse` y `ResponseBuilder`), `common.exception` y `common.handler.GlobalExceptionHandler`. También incluye un `DataSeeder` que genera datos de prueba con JavaFaker. Se mantuvo esa estructura original y el proyecto se compiló con `mvn clean package -DskipTests` y se ejecutó en el puerto 8080 (Figuras 2 a 4).

Figura 2

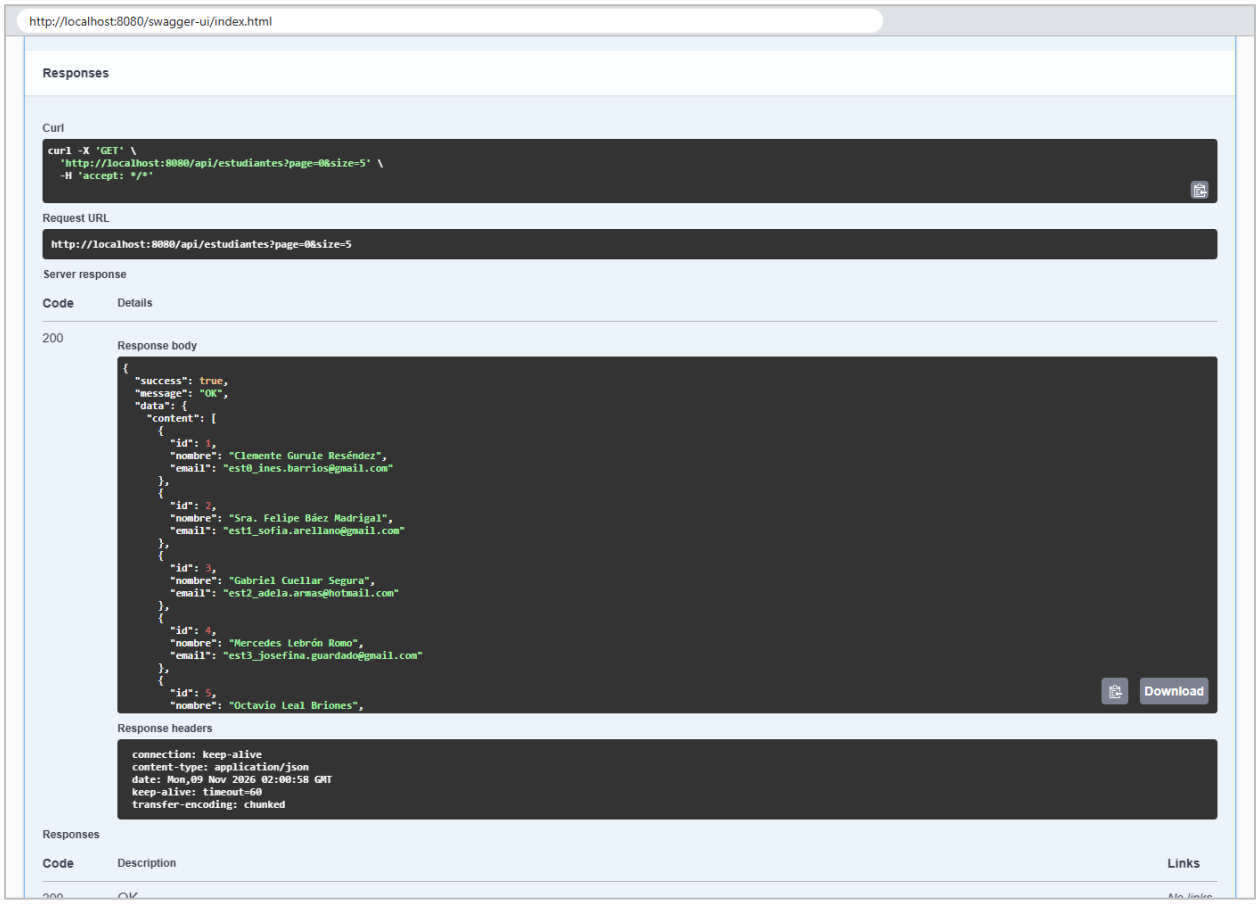
Swagger UI del proyecto base en el puerto 8080



Nota. Se observan los controladores de programas, estudiantes, asignaturas, grupos y matrículas.

Figura 3

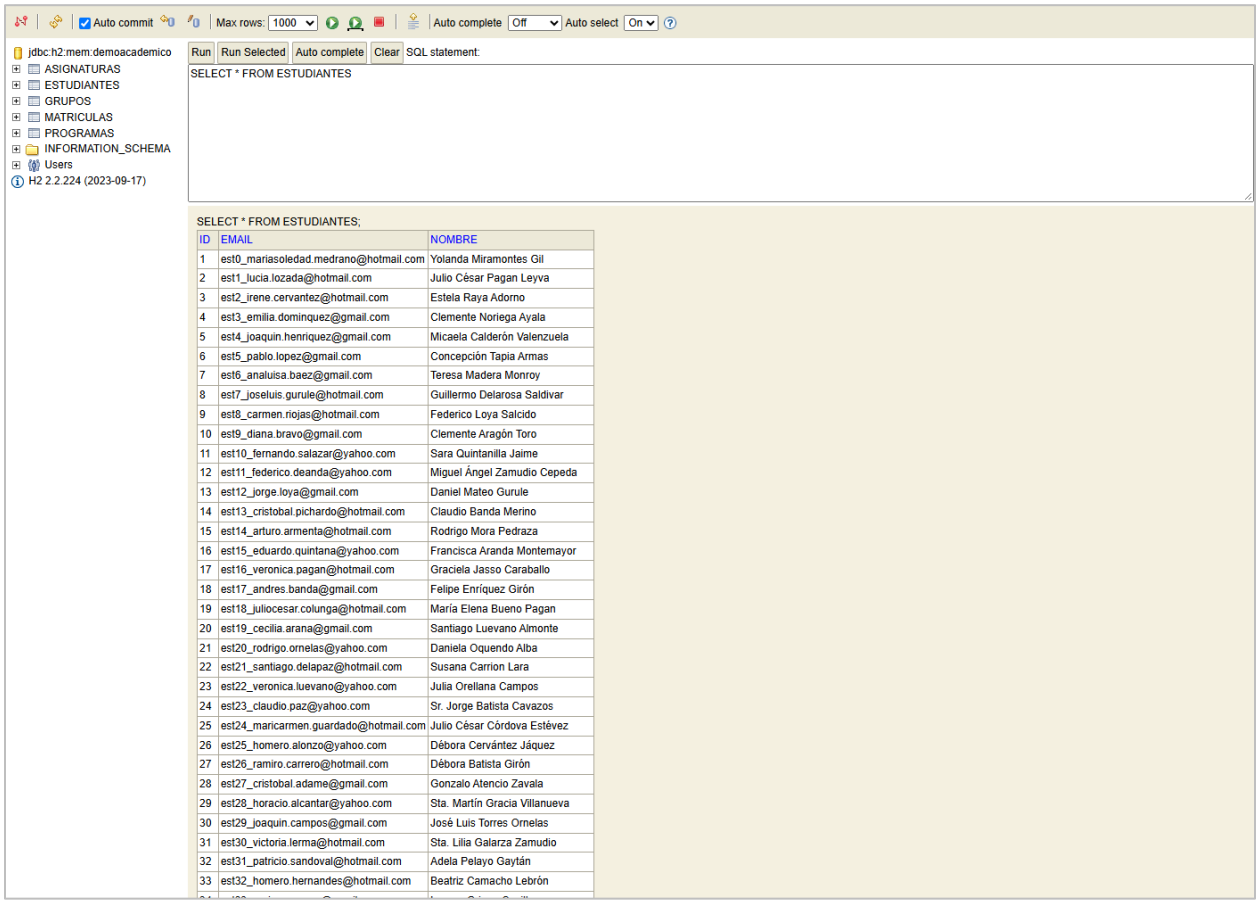
Consulta GET /api/estudiantes en el proyecto base



Nota. El proyecto base responde con código 200 y la estructura estándar `ApiResponse` sin requerir token.

Figura 4

Tabla ESTUDIANTES en la consola H2 del proyecto base



SELECT * FROM ESTUDIANTES;

ID	EMAIL	NOMBRE
1	est0_mariasoledad.medrano@hotmail.com	Yolanda Miramontes Gil
2	est1_lucia.lozada@hotmail.com	Julio César Pagan Leyva
3	est2_irene.cervantez@hotmail.com	Estela Raya Adorno
4	est3_emilia.dominquez@gmail.com	Clemente Noriega Ayala
5	est4_joaquin.henriquez@gmail.com	Micaela Calderón Valenzuela
6	est5_pablo.lopez@gmail.com	Concepción Tapia Armas
7	est6_analuisa.baez@gmail.com	Teresa Madera Monroy
8	est7_joseluis.gurule@hotmail.com	Guillermo Delarosa Saldivar
9	est8_carmen.riojas@hotmail.com	Federico Loya Salcido
10	est9_diana.bravo@gmail.com	Clemente Aragón Toro
11	est10_fernando.salazar@yahoo.com	Sara Quintanilla Jaime
12	est11_federico.deanda@yahoo.com	Miguel Ángel Zamudio Cepeda
13	est12_jorge.loya@gmail.com	Daniel Mateo Gurule
14	est13_cristobal.pichardo@hotmail.com	Claudio Banda Merino
15	est14_arturo.armenta@hotmail.com	Rodrigo Mora Pedraza
16	est15_eduardo.quintana@yahoo.com	Francisca Aranda Montemayor
17	est16_veronica.pagan@hotmail.com	Graciela Jasso Caraballo
18	est17_andres.banda@gmail.com	Felipe Enríquez Girón
19	est18_juliocesar.colunga@hotmail.com	María Elena Bueno Pagan
20	est19_cecilia.arana@gmail.com	Santiago Luevano Almonte
21	est20_rodrigo.ornelas@yahoo.com	Daniela Oquendo Alba
22	est21_santiago.delapaz@hotmail.com	Susana Carrion Lara
23	est22_veronica.luevano@yahoo.com	Julia Orellana Campos
24	est23_claudio.paz@yahoo.com	Sr. Jorge Batista Cavazos
25	est24_maricarmen.guardado@hotmail.com	Julio César Córdova Estévez
26	est25_homero.alonzo@yahoo.com	Débora Cervántez Jáquez
27	est26_ramiro.carrero@hotmail.com	Débora Batista Girón
28	est27_cristobal.adame@gmail.com	Gonzalo Atencio Zavala
29	est28_horacio.alcantar@yahoo.com	Sta. Martín Gracia Villanueva
30	est29_joaquin.campos@gmail.com	José Luis Torres Ornelas
31	est30_victoria.lerma@hotmail.com	Sta. Lilia Galarza Zamudio
32	est31_patricio.sandoval@hotmail.com	Adela Pelayo Gaytán
33	est32_homero.hernandes@hotmail.com	Beatriz Camacho Lebrón

Nota. Datos generados por el `DataSeeder` al iniciar la aplicación.

Parte 1: consulta-service a partir del proyecto base

Se creó la carpeta raíz `microservicios-demoacademico` y dentro de ella se copió el proyecto base con el nombre `consulta-service`. En el `pom.xml` se cambiaron `artifactId`, `name` y `description`, se agregó el bloque `dependencyManagement` con `spring-cloud-dependencies` 2023.0.1 y se incorporaron las dependencias `spring-cloud-starter-openfeign` y `spring-boot-starter-security`. En `application.yaml` el puerto pasó a 8082 y se añadió la propiedad `services.auth.url: http://localhost:8081`. Por último, la clase principal se anotó con `@EnableFeignClients`. El archivo `docker-compose.yaml` original del

repositorio, que usaba MySQL, se retiró de este servicio porque la orquestación se hace desde la carpeta raíz con H2 embebido.

Parte 2: auth-service con H2, BCrypt y JWT

Se creó el proyecto `auth-service` (paquete `co.edu.authservice`, puerto 8081) con las clases indicadas en la guía: la entidad `AuthUser` con restricción única sobre `username`; el repositorio `AuthUserRepository`; los DTO `LoginRequest` y `LoginResponse`; `TokenUtils`, que elimina el prefijo *Bearer* repetido; `JwtUtil`, que genera y valida tokens con la librería JJWT 0.11.5; el servicio `AuthService`, que compara la contraseña con `PasswordEncoder.matches`; `SecurityConfig`, que publica el `BCryptPasswordEncoder` y deja libres `/auth/**`, Swagger y H2; `AuthDataSeeder`, que registra los usuarios de prueba de la Tabla 1; y `AuthController`, con los endpoints `POST /auth/login` y `GET /auth/validate`.

Tabla 1

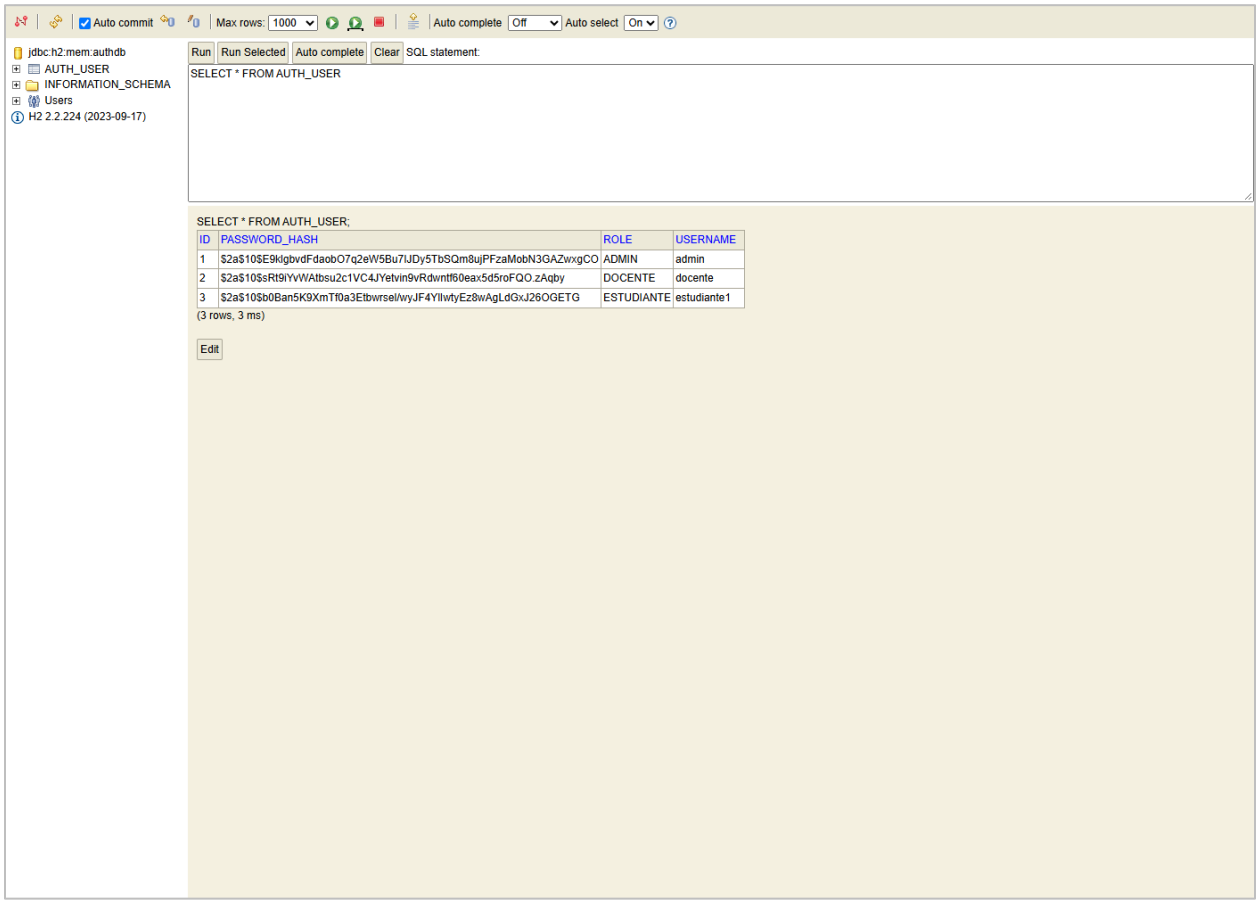
Usuarios de prueba registrados por AuthDataSeeder

Usuario	**Contraseña**	**Rol**
admin	Admin2026*	ADMIN
docente	Docente2026*	DOCENTE
estudiante1	Estu2026*	ESTUDIANTE

La Figura 5 confirma que en la tabla `AUTH_USER` solo se almacena el resumen BCrypt de cada contraseña.

Figura 5

Tabla AUTH_USER en la consola H2 de auth-service



Nota. La columna `PASSWORD_HASH` contiene resúmenes BCrypt con costo 10; ninguna contraseña está en texto plano.

Parte 3: integración con OpenFeign

En `consulta-service` se agregaron el DTO `TokenValidationResponse`, que refleja la respuesta de `/auth/validate` (`valid`, `username`, `role` y `message`); la interfaz `AuthFeignClient`, anotada con `@FeignClient(name = "authClient", url = "${services.auth.url}")`; una copia de `TokenUtils`; y `TokenValidationService`, que normaliza el token, invoca al cliente Feign y devuelve `true` solo cuando la respuesta indica

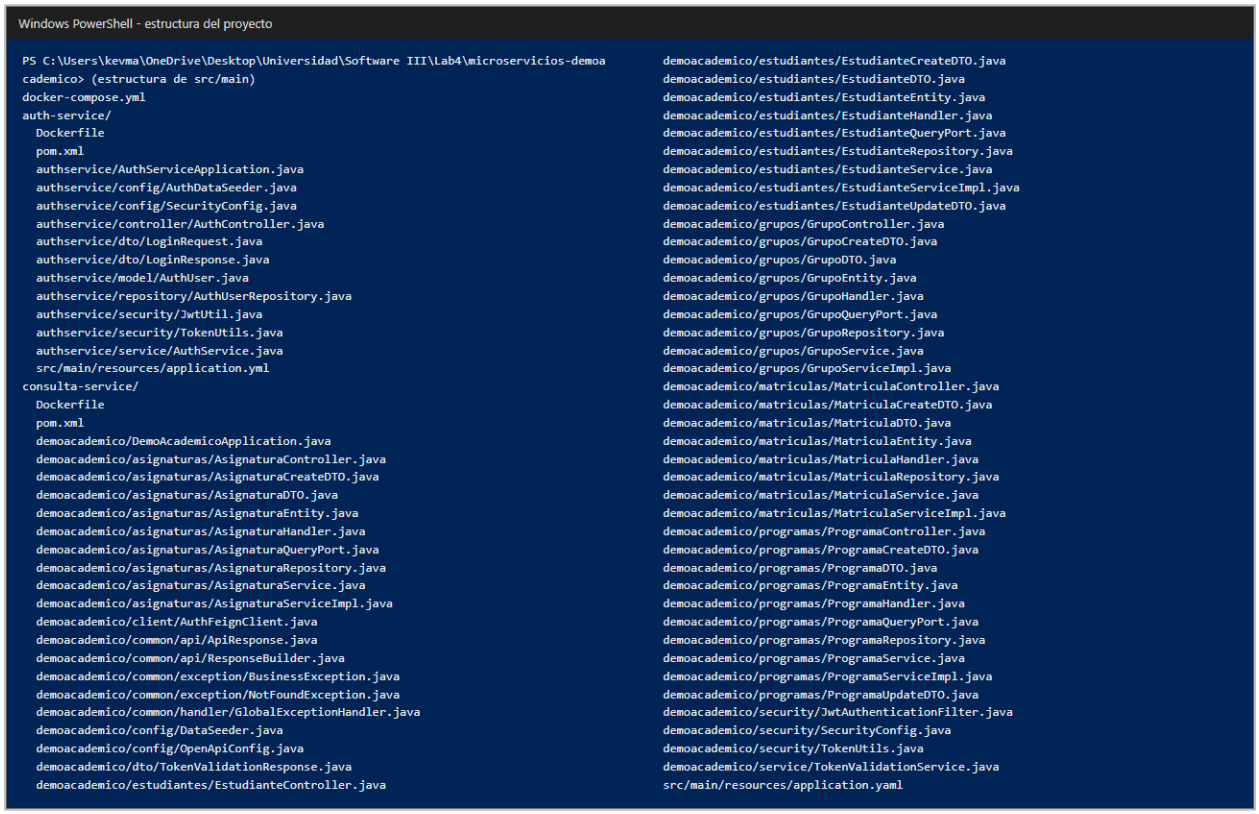
`valid = true`. Cualquier excepción, por ejemplo un 401 de *auth-service* o un error de conexión, se interpreta como token no válido.

Parte 4: seguridad del flujo de acceso en consulta-service

Se creó `OpenApiConfig`, que registra el esquema de seguridad `bearerAuth` de tipo HTTP *bearer* con formato JWT. Con él aparece el botón *Authorize* en Swagger. El `JwtAuthenticationFilter` extiende `OncePerRequestFilter`: omite Swagger, la documentación OpenAPI, la consola H2 y la creación de estudiantes (`POST /api/estudiantes`, tal como lo define la guía), y para el resto de rutas que empiezan por `/api/estudiantes` exige el encabezado `Authorization`. Si falta el token responde 401 con el mensaje "Token no enviado o formato inválido"; si *auth-service* no lo valida responde 401 con "Token inválido o expirado". `SecurityConfig` desactiva CSRF y los encabezados de marcos para H2, y registra el filtro antes de `UsernamePasswordAuthenticationFilter` dentro de la cadena de filtros de Spring Security (Spring, s.f.-a). Además, `EstudianteController` se anotó con `@SecurityRequirement(name = "bearerAuth")`. La Figura 6 resume la estructura final de ambos servicios.

Figura 6

Estructura final de auth-service y consulta-service



Nota. Se listan los archivos de `src/main`. En *consulta-service* se conservan los módulos del repositorio base y se agregan los paquetes `client`, `dto`, `security` y `service`.

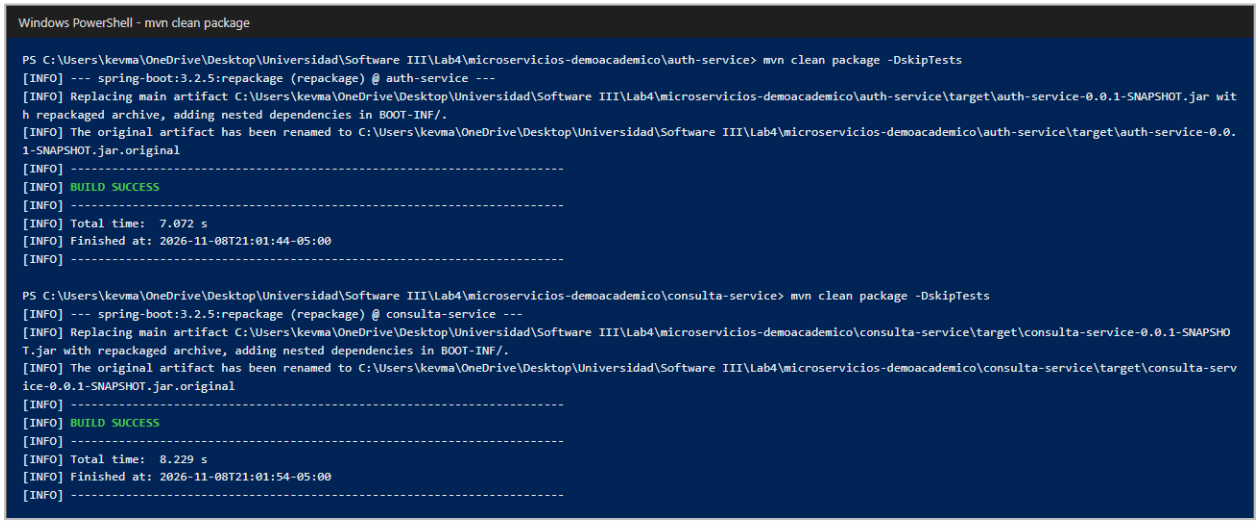
Parte 5: Dockerización y orquestación

Cada servicio tiene un `Dockerfile` basado en `eclipse-temurin:21-jdk` que copia el JAR generado por Maven y lo ejecuta con `java -jar`, exponiendo los puertos 8081 y 8082. El archivo `docker-compose.yml` de la carpeta raíz construye ambas imágenes, las conecta en la red `micro-net` y, mediante la variable `SERVICES_AUTH_URL=http://auth-service:8081`, hace que *consulta-service* llame a *auth-service* por el nombre del servicio y por su puerto interno, no por `localhost`. Se omitió el atributo `version` porque Docker Compose v2 lo considera

obsoleto. Las Figuras 7 y 8 muestran la compilación de ambos proyectos y los contenedores en ejecución.

Figura 7

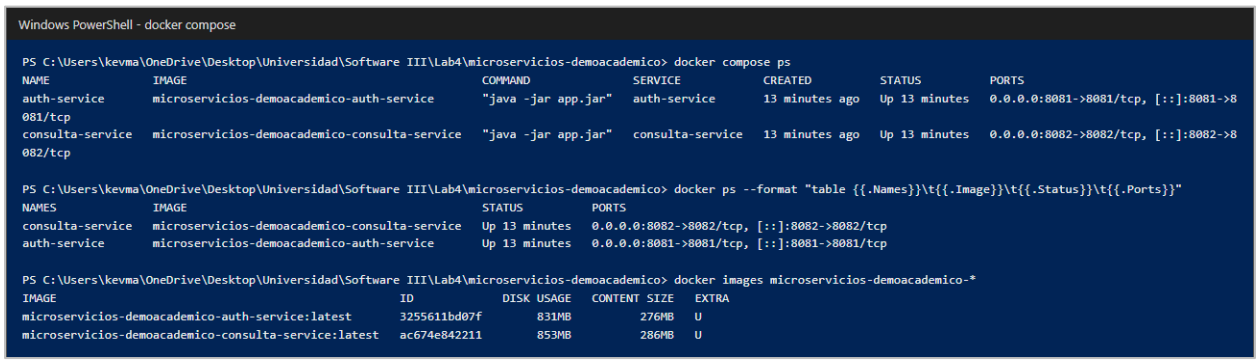
Compilación de auth-service y consulta-service con Maven



Nota. Ambos proyectos terminan con `BUILD SUCCESS` y generan su JAR ejecutable.

Figura 8

Contenedores levantados con docker compose up --build



Nota. Los contenedores `auth-service` y `consulta-service` están activos y publican los puertos 8081 y 8082.

Pruebas obligatorias en Swagger

Las pruebas se ejecutaron con los dos servicios corriendo en Docker y siguiendo el orden obligatorio de la guía: inicio de sesión en *auth-service*, copia del token, autorización en el

Swagger de *consulta-service* y consumo del endpoint protegido. La Tabla 2 resume el guion mínimo y los resultados obtenidos.

Tabla 2

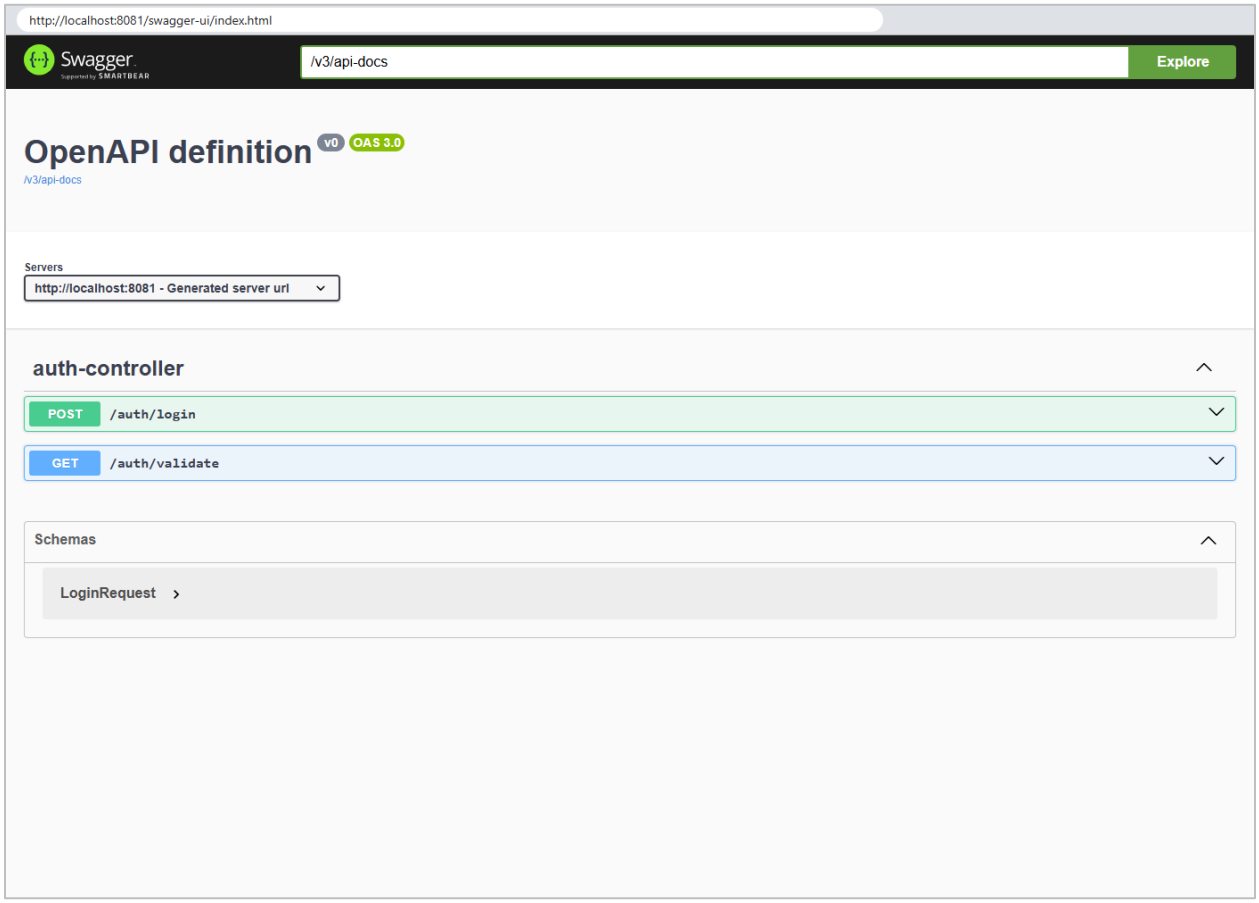
Resultados del guion mínimo de pruebas

Caso	Petición	Resultado obtenido	Figura
1. Login exitoso	POST /auth/login (admin / Admin2026*)	200 con token JWT de tipo Bearer	10 y 11
2. Login fallido	POST /auth/login con contraseña incorrecta	401 "Credenciales inválidas"	12
3. Validación manual	GET /auth/validate?token=...	200 {valid: true, username: admin, role: ADMIN}	13
4. Con token válido	GET /api/estudiantes con Authorize	200 con la lista paginada de estudiantes	17
5. Sin autorizar	GET /api/estudiantes sin Authorize	401 "Token no enviado o formato inválido"	18
6. Token inválido	GET /api/estudiantes con token alterado	401 "Token inválido o expirado"	19

Inicio de sesión exitoso

Figura 9

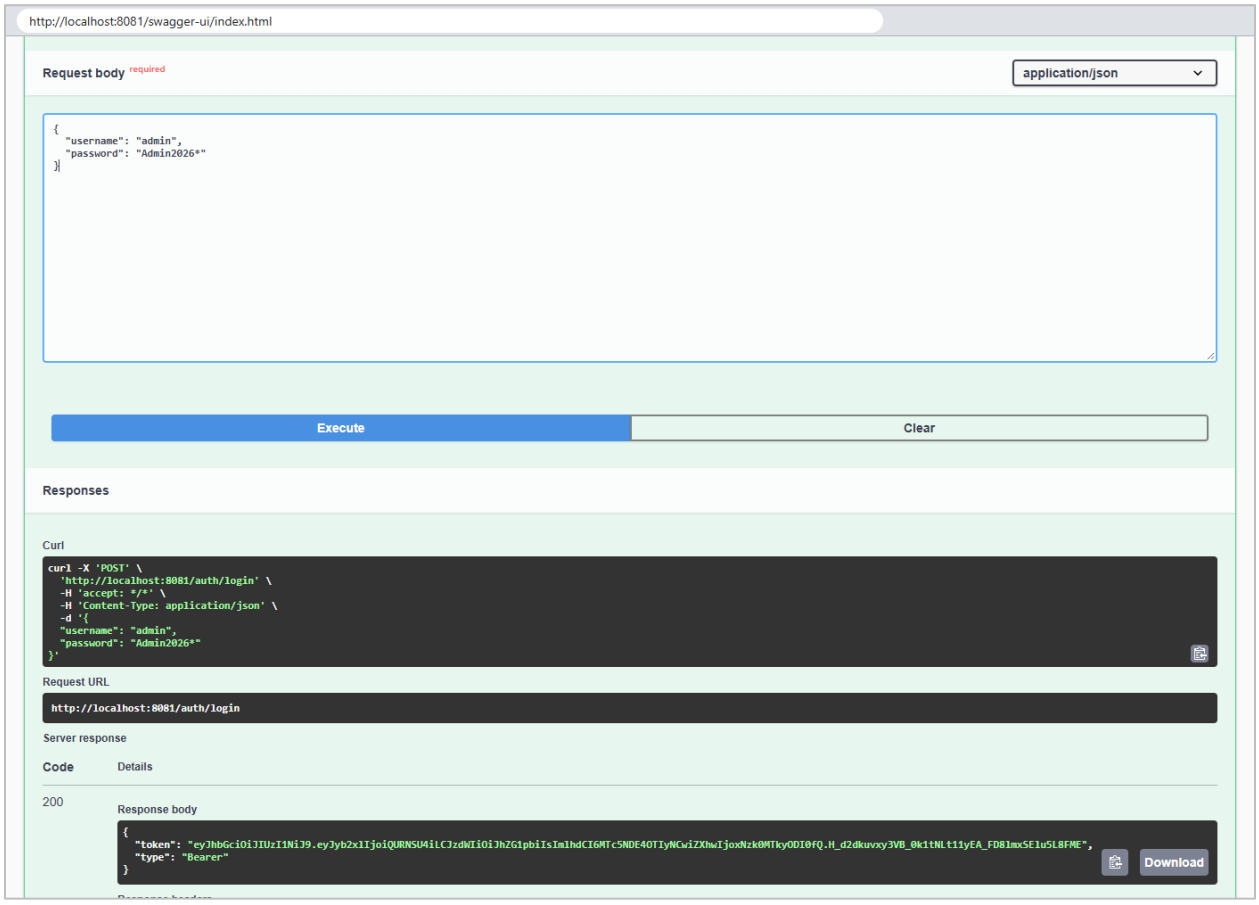
Swagger UI de auth-service



Nota. Se publican los endpoints `POST /auth/login` y `GET /auth/validate` en el puerto 8081.

Figura 10

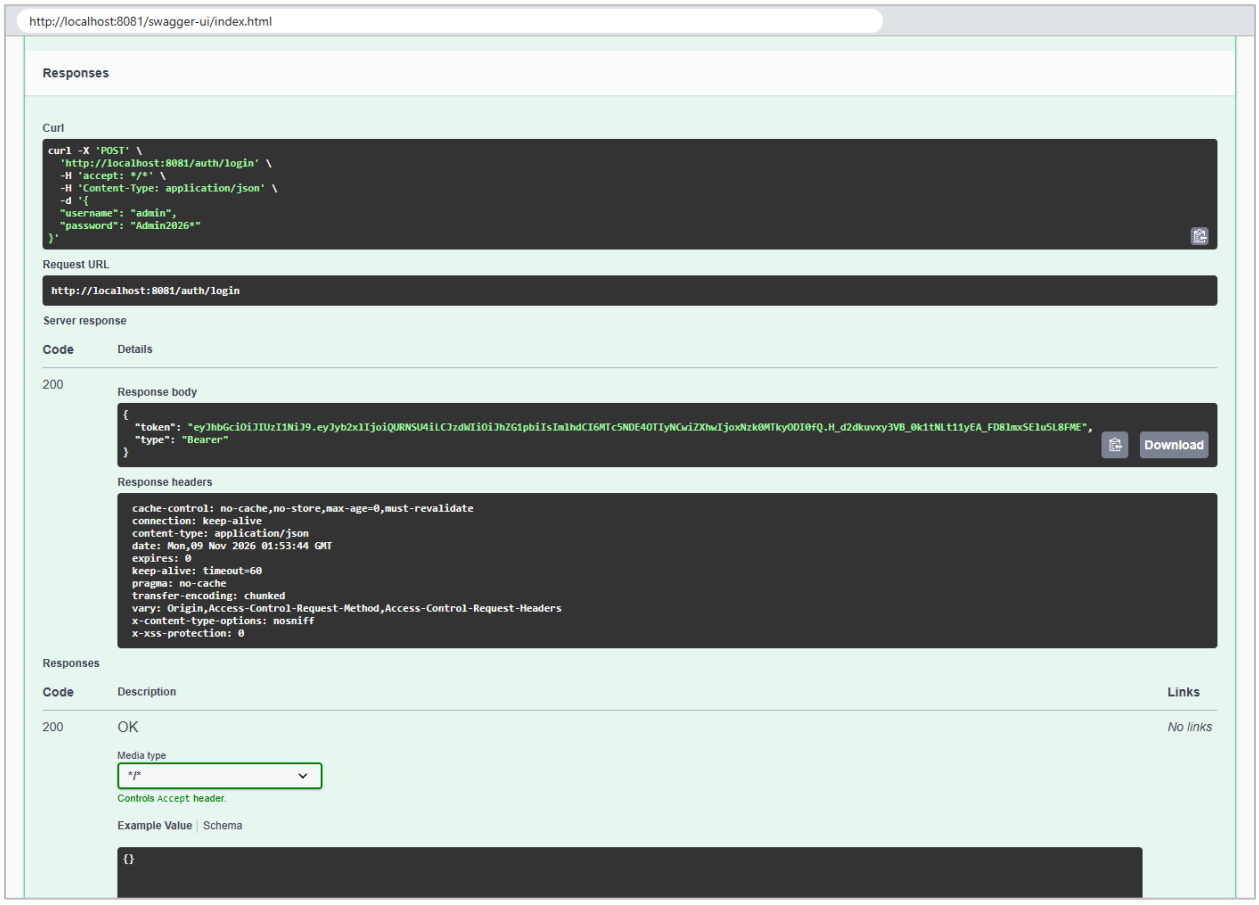
Petición de inicio de sesión con el usuario admin



Nota. Cuerpo enviado desde *Try it out* con las credenciales válidas del administrador.

Figura 11

Respuesta del inicio de sesión exitoso

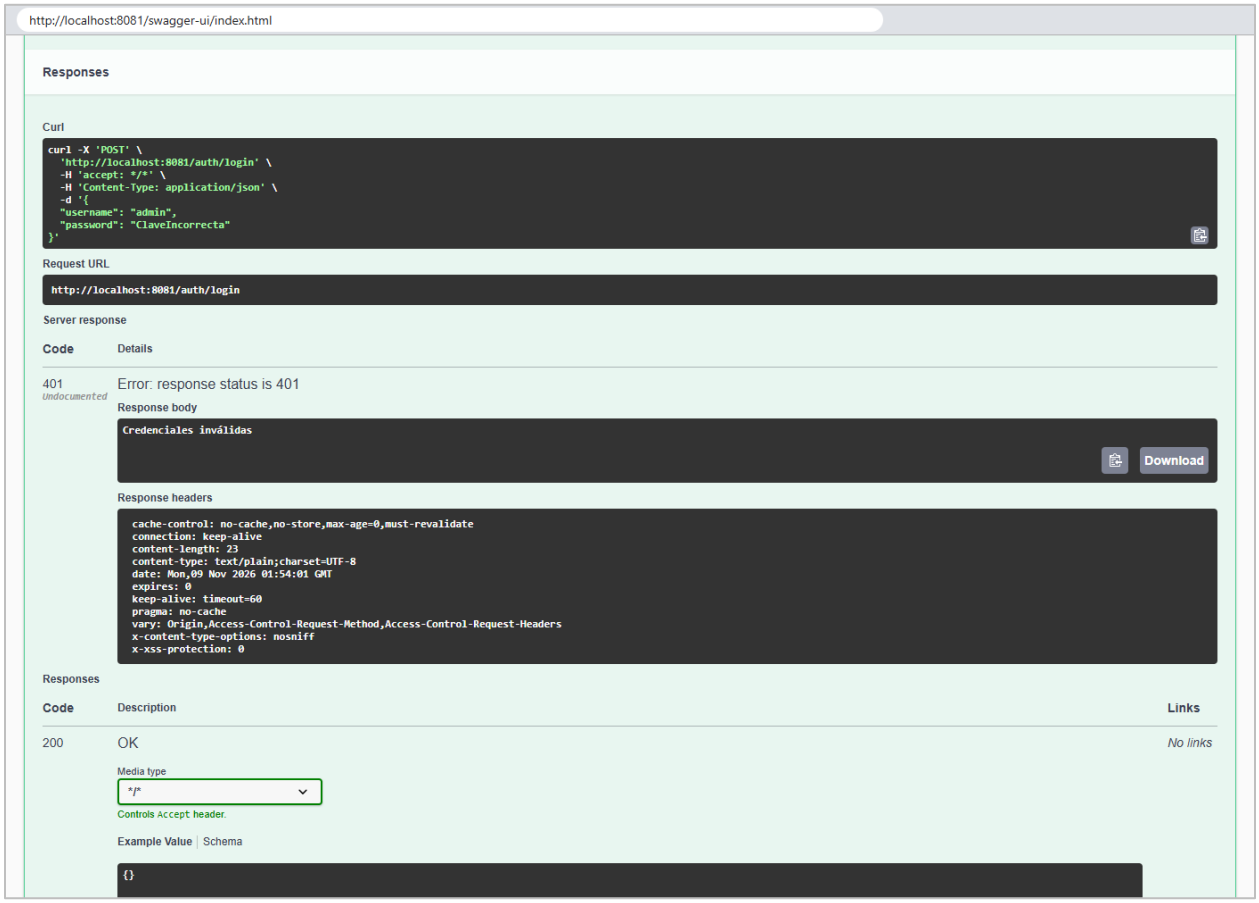


Nota. El servicio responde 200 con el campo `token` (JWT firmado con HS256) y `type: Bearer`.

Inicio de sesión fallido

Figura 12

Inicio de sesión con contraseña incorrecta

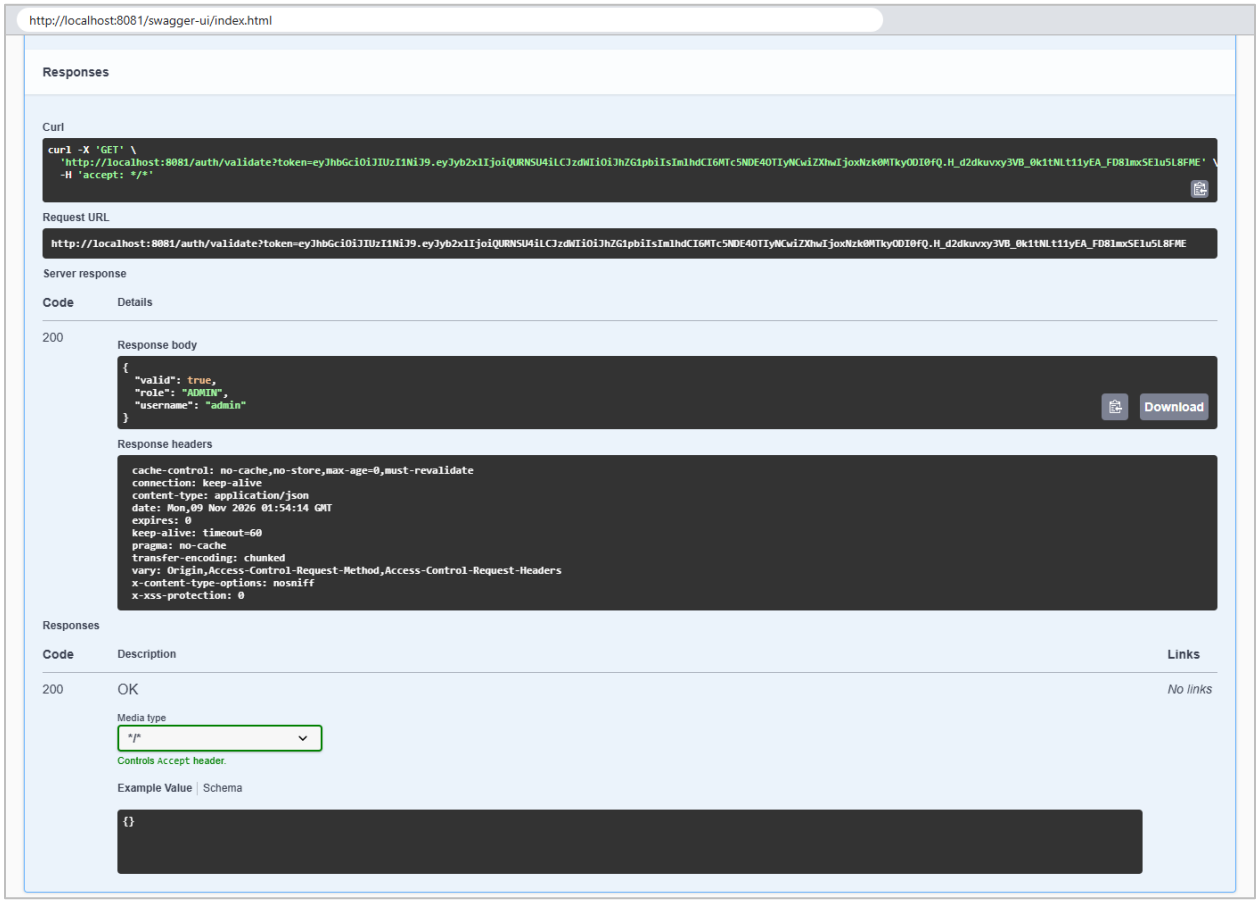


Nota. BCrypt no encuentra coincidencia y el servicio responde 401 con el mensaje "Credenciales inválidas".

Validación manual del token

Figura 13

Validación manual del token en GET /auth/validate

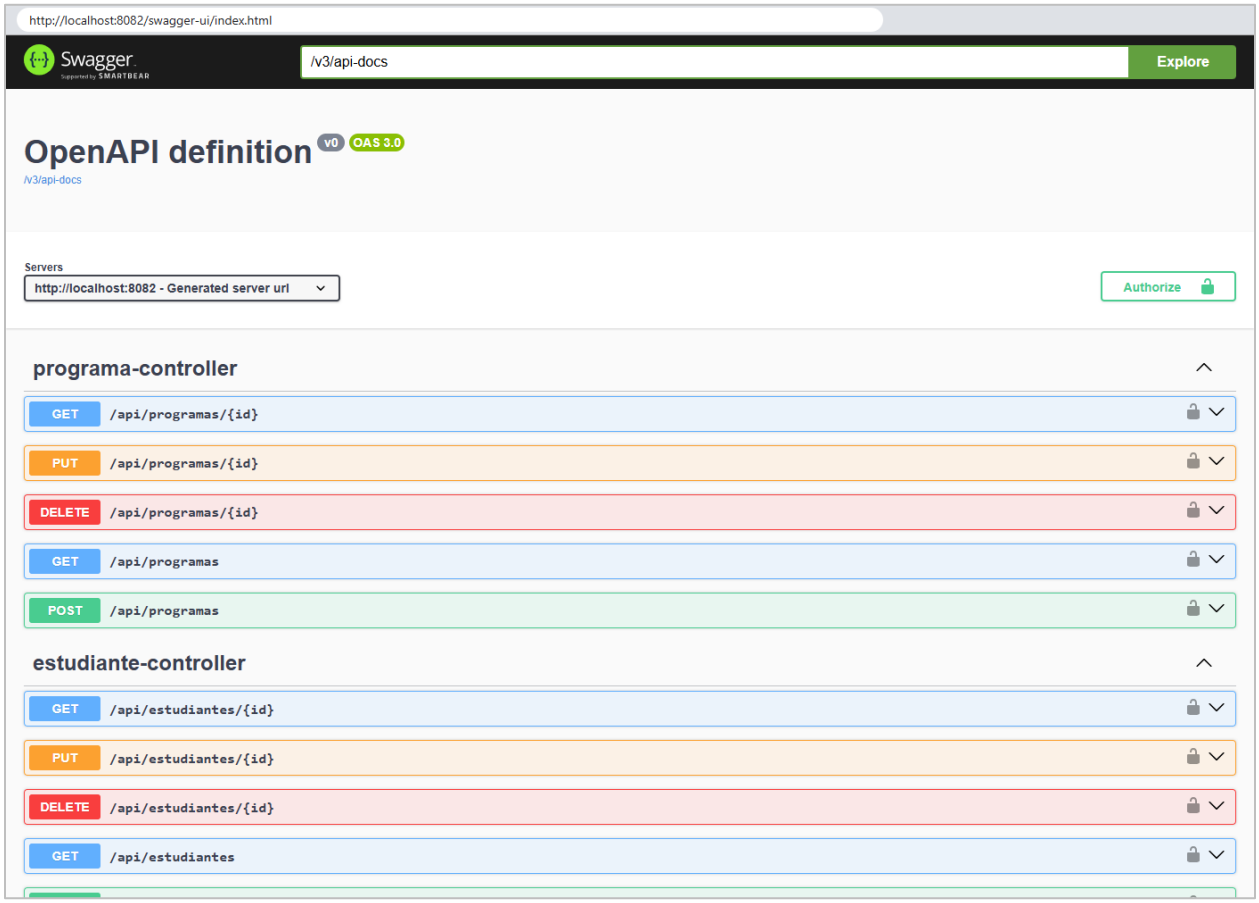


Nota. Se pegó únicamente el valor del token, sin el prefijo *Bearer*; la respuesta confirma usuario y rol.

Uso del botón Authorize en consulta-service

Figura 14

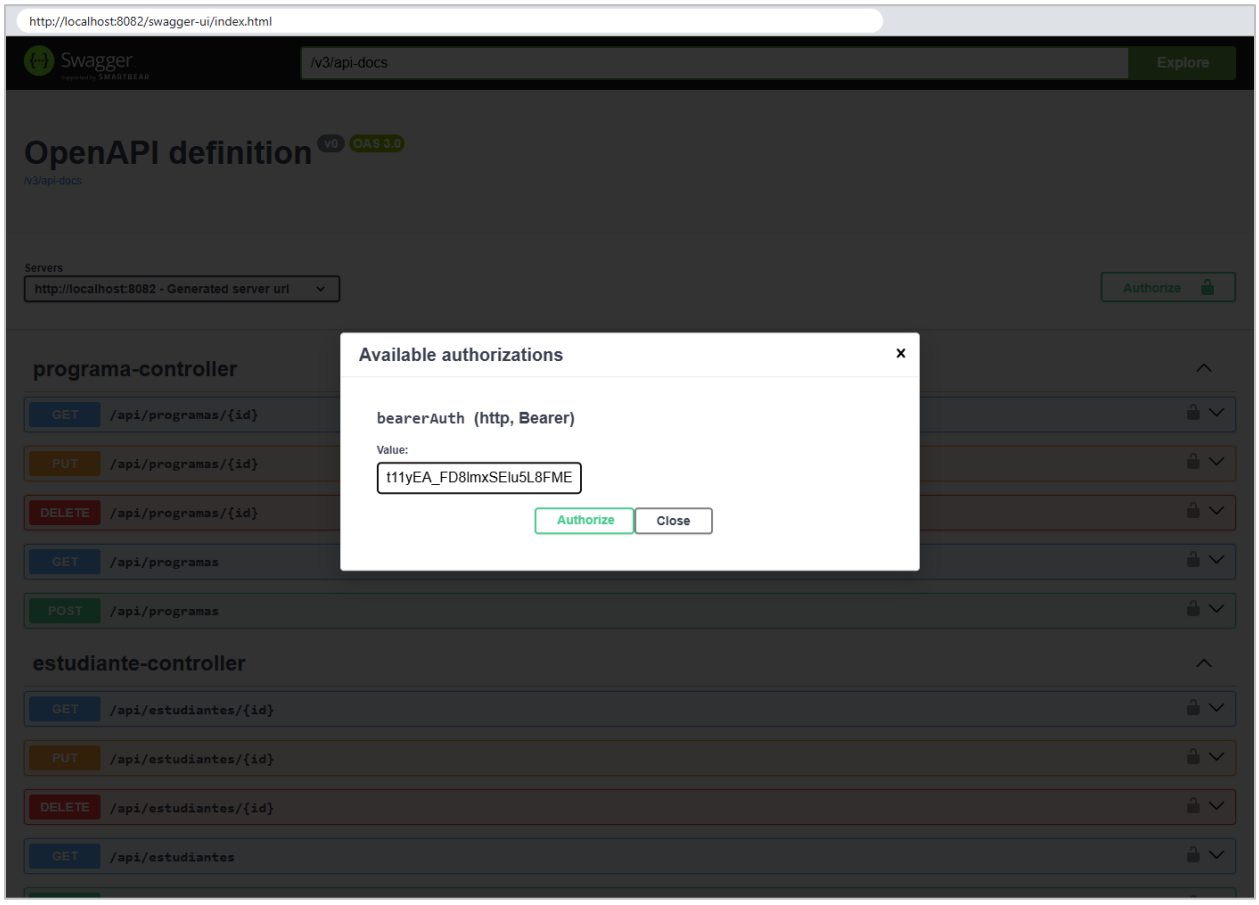
Swagger UI de consulta-service con el botón Authorize



Nota. Los endpoints muestran un candado porque están documentados con el esquema `bearerAuth`.

Figura 15

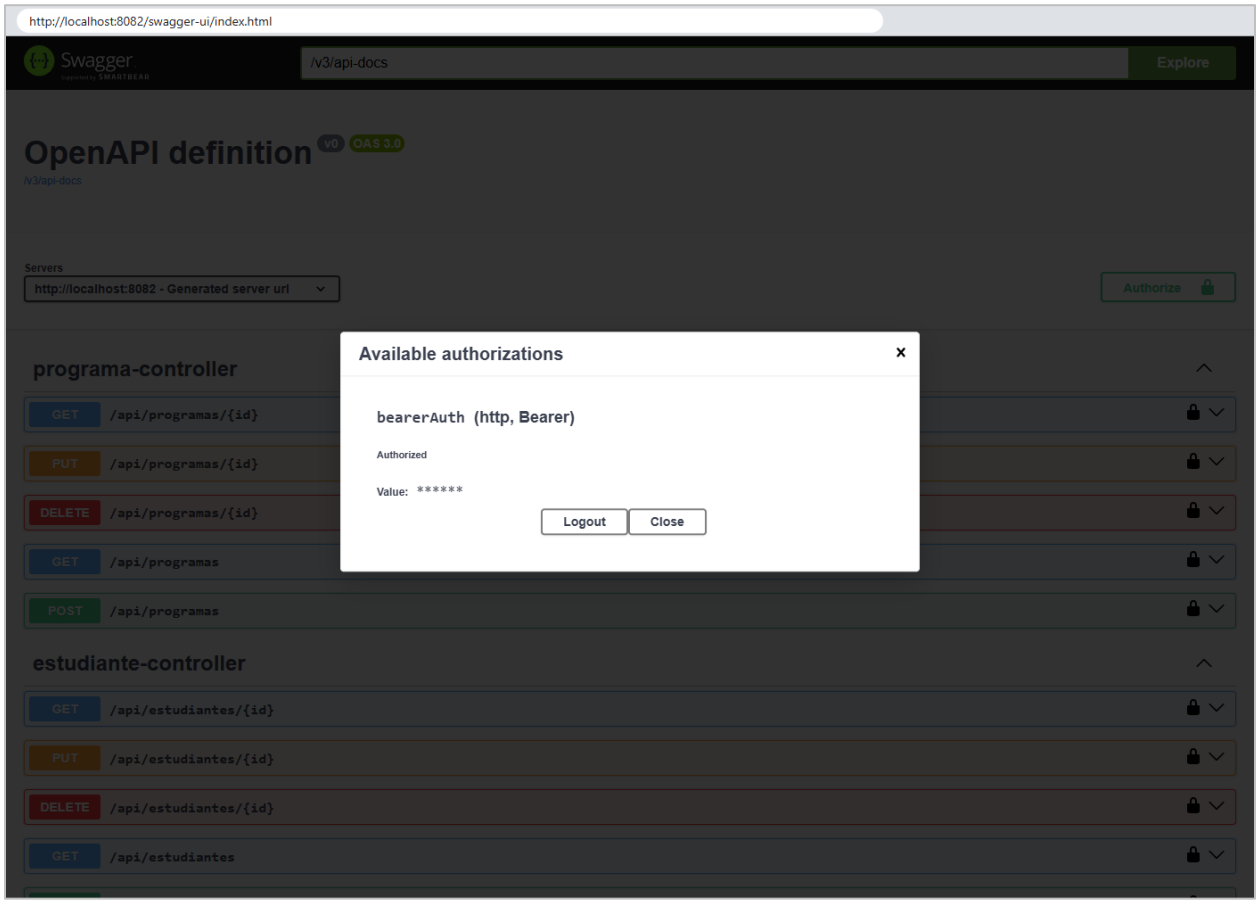
Token pegado en la ventana Available authorizations



Nota. Se pega solo el token, sin escribir la palabra *Bearer*, porque Swagger la agrega automáticamente.

Figura 16

Autorización aplicada en Swagger

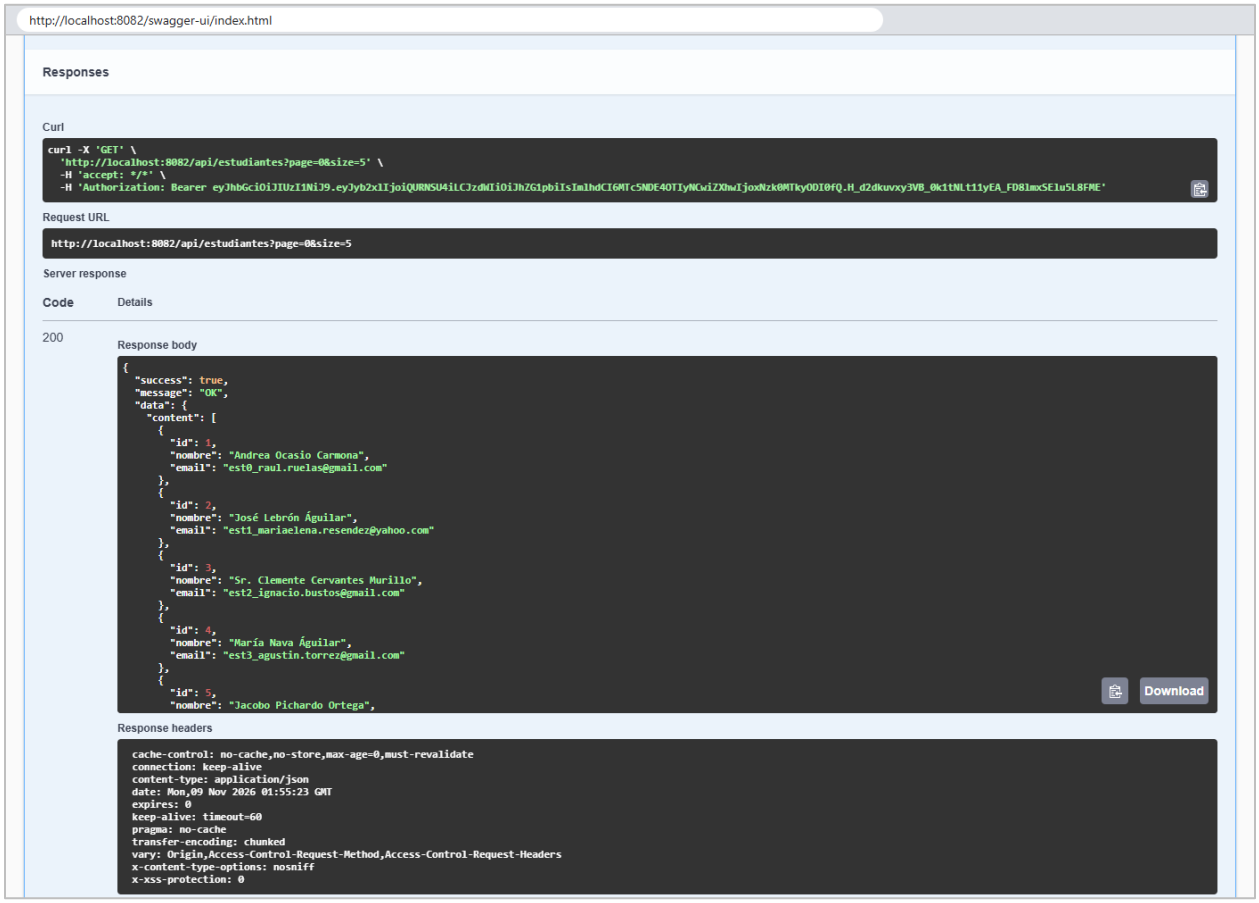


Nota. El esquema `bearerAuth` queda en estado *Authorized*.

Consulta de estudiantes con token válido

Figura 17

GET /api/estudiantes con token válido

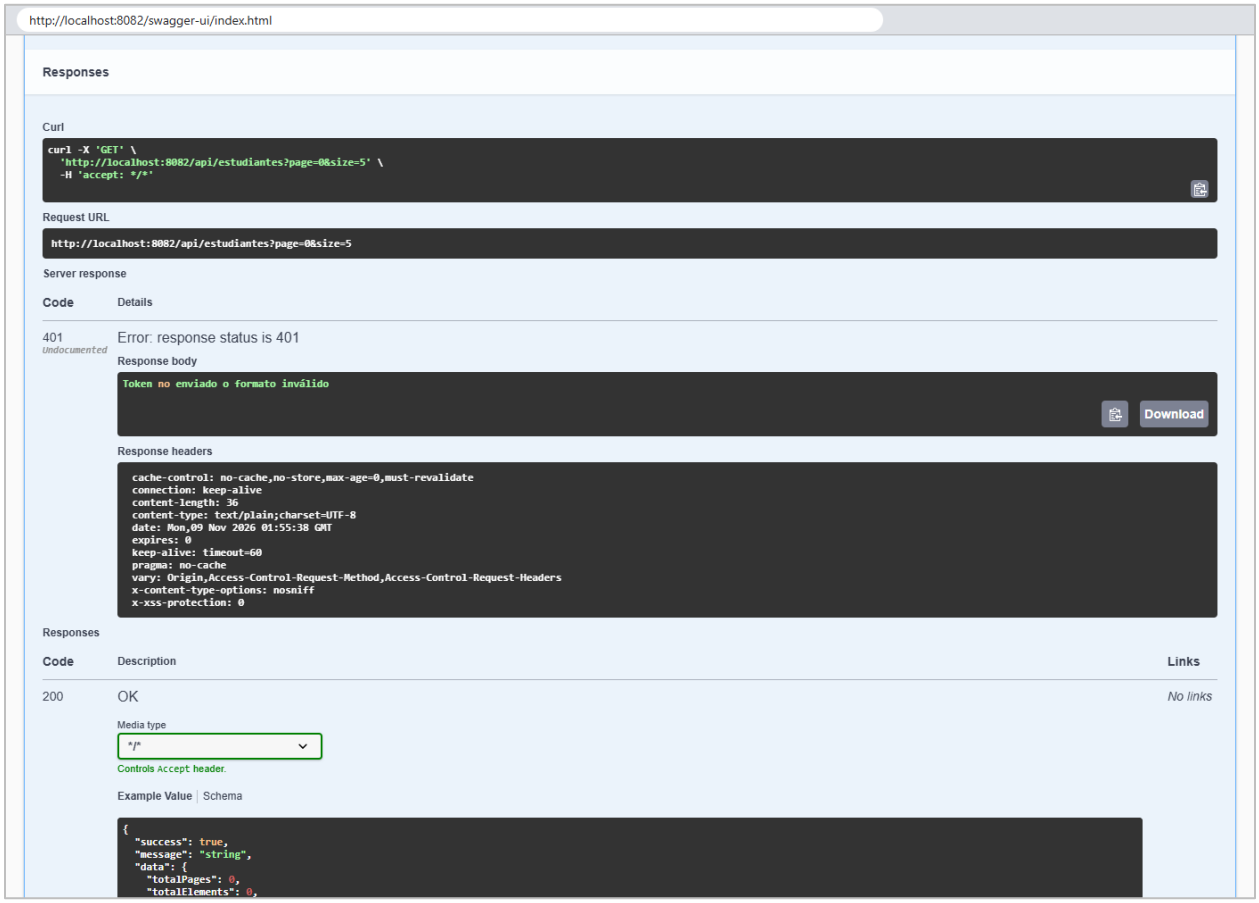


Nota. El comando curl generado incluye `Authorization: Bearer <token>` y la respuesta es 200 con los estudiantes paginados (`page=0`, `size=5`).

Consulta de estudiantes sin autorizar

Figura 18

GET /api/estudiantes sin autorizar en Swagger

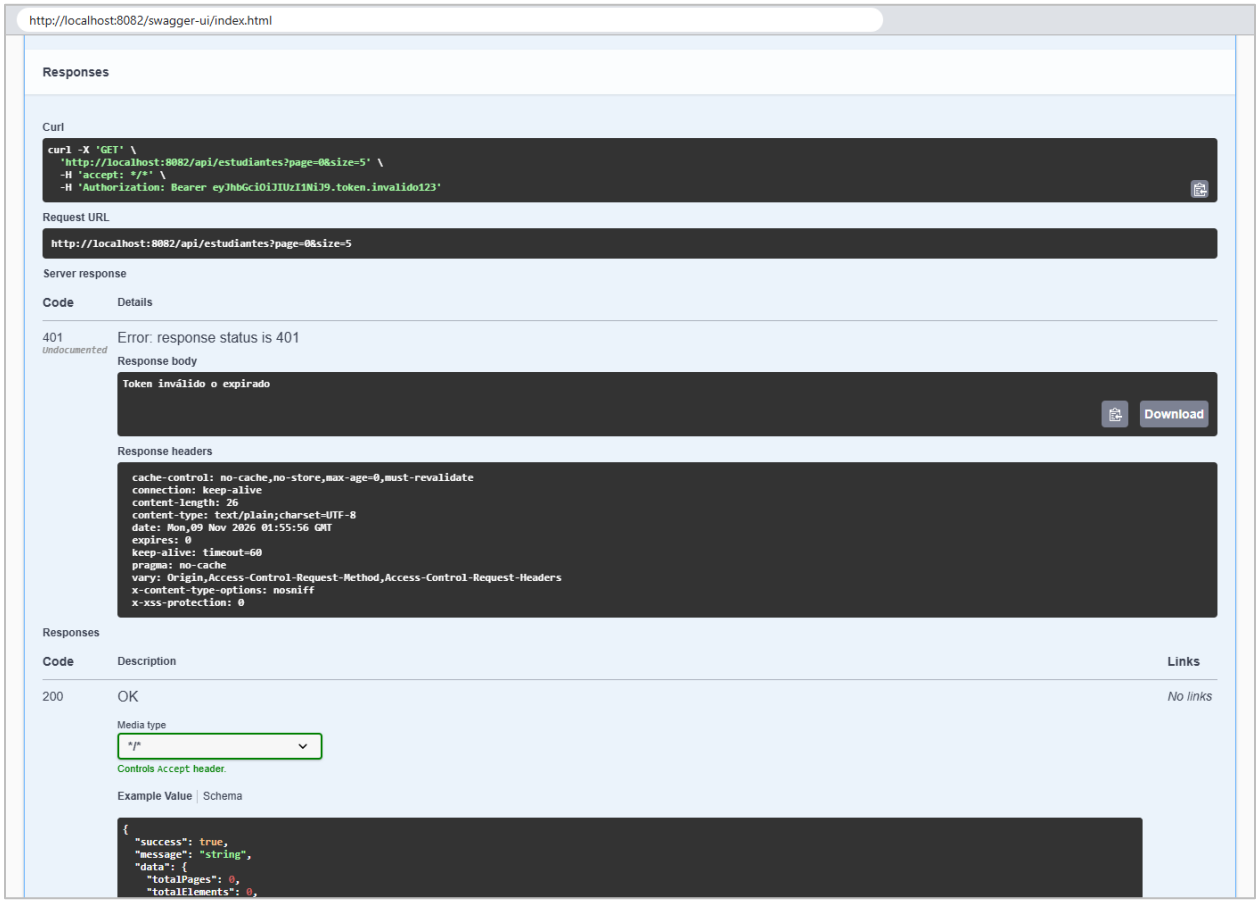


Nota. La petición no lleva encabezado `Authorization` y el filtro responde 401 "Token no enviado o formato inválido".

Consulta de estudiantes con token inválido

Figura 19

GET /api/estudiantes con token inválido

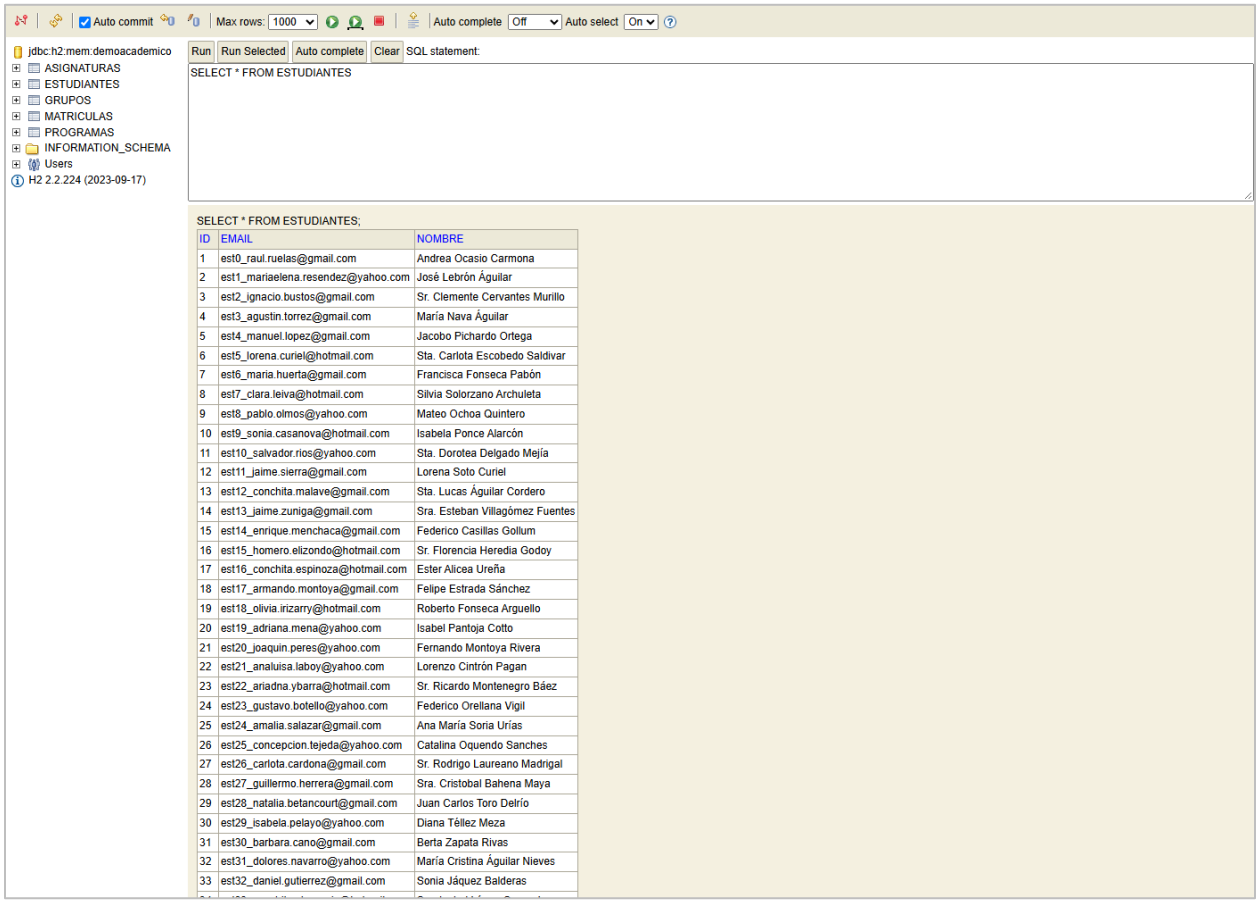


Nota. consulta-service consulta a *auth-service* mediante OpenFeign; como la firma no es válida, responde 401 "Token inválido o expirado".

Verificación de datos en H2 Console de consulta-service

Figura 20

Tabla ESTUDIANTES en la consola H2 de consulta-service



ID	EMAIL	NOMBRE
1	est0_raul.ruelas@gmail.com	Andrea Ocasio Carmona
2	est1_mariaelena.resendez@yahoo.com	José Lebrón Águilar
3	est2_ignacio.bustos@gmail.com	Sr. Clemente Cervantes Murillo
4	est3_agustin.torrez@gmail.com	María Nava Águilar
5	est4_manuel.lopez@gmail.com	Jacobo Pichardo Ortega
6	est5_lorena.curiel@hotmail.com	Sta. Carlota Escobedo Saldivar
7	est6_maria.huerta@gmail.com	Francisca Fonseca Pabón
8	est7_clara.leiva@hotmail.com	Silvia Solorzano Archuleta
9	est8_pablo.olmos@yahoo.com	Mateo Ochoa Quintero
10	est9_sonia.casanova@hotmail.com	Isabela Ponce Alarcón
11	est10_salvador.rios@yahoo.com	Sta. Dorotea Delgado Mejía
12	est11_jaime.sierra@gmail.com	Lorena Soto Curiel
13	est12_conchita.malave@gmail.com	Sta. Lucas Águilar Cordero
14	est13_jaime.zuniga@gmail.com	Sra. Esteban Villagómez Fuentes
15	est14_enrique.menchaca@gmail.com	Federico Casillas Gollum
16	est15_homero.elizondo@hotmail.com	Sr. Florencia Heredia Godoy
17	est16_conchita.espinoza@hotmail.com	Ester Alicea Ureña
18	est17_armando.montoya@gmail.com	Felipe Estrada Sánchez
19	est18_olivia.irizarry@hotmail.com	Roberto Fonseca Arguello
20	est19_adriana.mena@yahoo.com	Isabel Pantoja Cotto
21	est20_joaquin.peres@yahoo.com	Fernando Montoya Rivera
22	est21_analuisa.laboy@yahoo.com	Lorenzo Cintrón Pagan
23	est22_ariadna.ybarra@hotmail.com	Sr. Ricardo Montenegro Báez
24	est23_gustavo.botello@yahoo.com	Federico Orellana Vigil
25	est24_amalia.salazar@gmail.com	Ana María Soria Urías
26	est25_concepcion.tejeda@yahoo.com	Catalina Oquendo Sanches
27	est26_carlota.cardona@gmail.com	Sr. Rodrigo Laureano Madrigal
28	est27_guillermo.herrera@gmail.com	Sra. Cristobal Bahena Maya
29	est28_natalia.betancourt@gmail.com	Juan Carlos Toro Delrío
30	est29_isabela.pelayo@yahoo.com	Diana Téllez Meza
31	est30_barbara.cano@gmail.com	Berta Zapata Rivas
32	est31_dolores.navarro@yahoo.com	María Cristina Águilar Nieves
33	est32_daniel.gutierrez@gmail.com	Sonia Jáquez Balderas

Nota. Consola disponible en `http://localhost:8082/h2-console` con la URL JDBC `jdbc:h2:mem:demoacademico`.

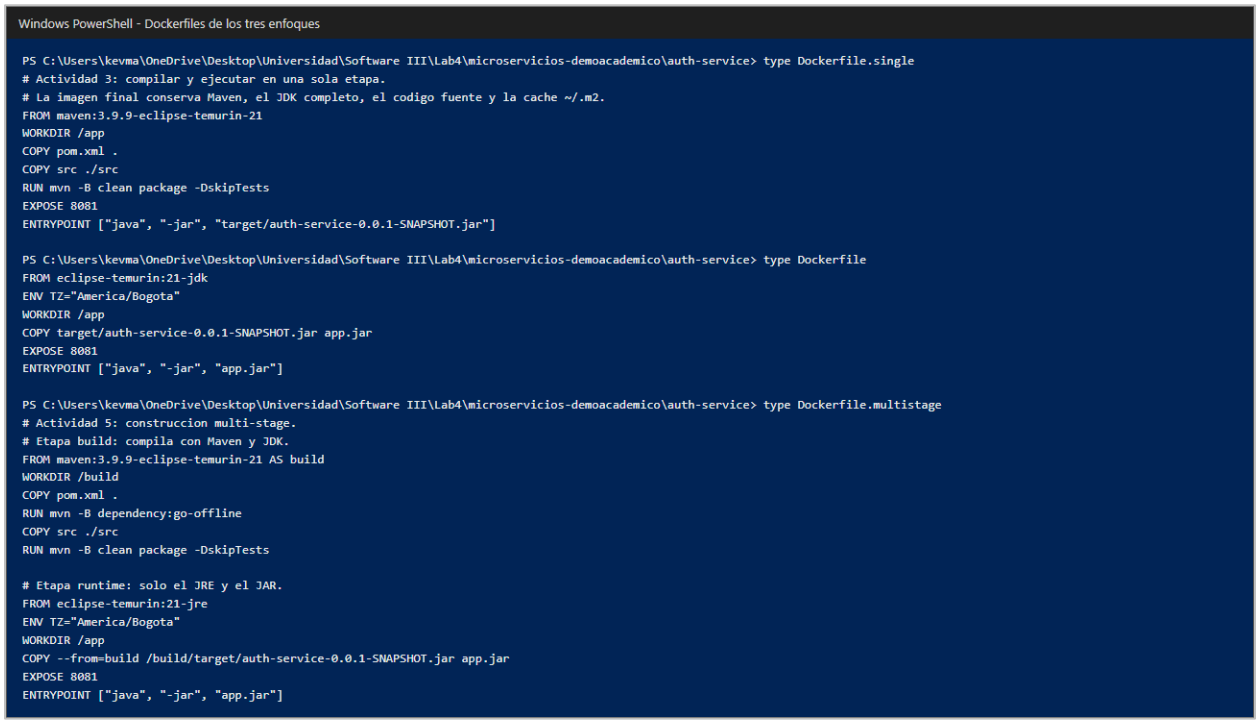
Actividades de contenerización: imágenes, volúmenes y redes

Para responder las actividades se construyó la imagen de *auth-service* con los tres enfoques propuestos y se midieron sus resultados en el mismo equipo. Las tres variantes se encuentran en la carpeta `auth-service`: `Dockerfile.single` compila dentro del Dockerfile, `Dockerfile` copia el JAR generado previamente (compilación separada) y

`Dockerfile.multistage` usa dos etapas. Las imágenes se etiquetaron como `lab04-auth:single`, `lab04-auth:jar` y `lab04-auth:multistage` (Figura 21).

Figura 21

Dockerfiles de los tres enfoques de construcción



Nota. De arriba hacia abajo: compilación dentro del Dockerfile, compilación separada y multi-stage.

Actividad 1: proyecto, endpoint y nombre de las imágenes

1. ¿Trabajará con un proyecto existente o con uno creado para el laboratorio? Con un proyecto existente: el repositorio *demo-academico-application* del curso (ojugama, s.f.), que se reutilizó como *consulta-service*, más el servicio *auth-service* creado para este laboratorio (Figuras 2 y 6).

2. ¿Qué endpoint simple expone su aplicación? *auth-service* expone `POST /auth/login` y `GET /auth/validate`; *consulta-service* expone `GET /api/estudiantes`. Para

verificar el arranque de los contenedores se usó `GET /v3/api-docs`, que responde sin autenticación.

3. ¿Cuál será el nombre base que usará para sus imágenes Docker? Para la comparación se usó `lab04-auth` con las etiquetas `single`, `jar` y `multistage`. En la solución integrada, Docker Compose nombra las imágenes `microservicios-demoacademico-auth-service` y `microservicios-demoacademico-consulta-service`.

Actividad 2: validación previa del proyecto

1. ¿Por qué es importante validar el proyecto antes de contenerizarlo? Porque Docker solo empaqueta lo que recibe. Si el código no compila o la aplicación no arranca, el error aparece tarde, dentro del build de la imagen o del contenedor, donde es más difícil de diagnosticar. Validar primero con `mvn clean package` y ejecutar el JAR localmente separa los errores de código de los errores de contenerización.

2. ¿Qué archivo ejecutable genera Spring Boot después del build? Un JAR ejecutable (*fat jar*) en la carpeta `target`: `auth-service-0.0.1-SNAPSHOT.jar` y `consulta-service-0.0.1-SNAPSHOT.jar`, de unos 55 MB cada uno. El plugin `spring-boot-maven-plugin` lo reempaqueta con todas las dependencias y el servidor Tomcat embebido, por lo que se ejecuta con `java -jar` (Figura 7).

3. ¿Qué puerto usa la aplicación? *auth-service* usa el 8081 y *consulta-service* el 8082, definidos en `server.port` y publicados con `EXPOSE` y `ports` en Docker Compose. El proyecto base usa el 8080.

Actividad 3: compilar dentro del Dockerfile

1. ¿Qué herramientas de build quedan dentro de la imagen? Maven 3.9.9 y el JDK completo, con el compilador `javac`, porque la imagen parte de `maven:3.9.9-eclipse-temurin-21` (Figura 22).

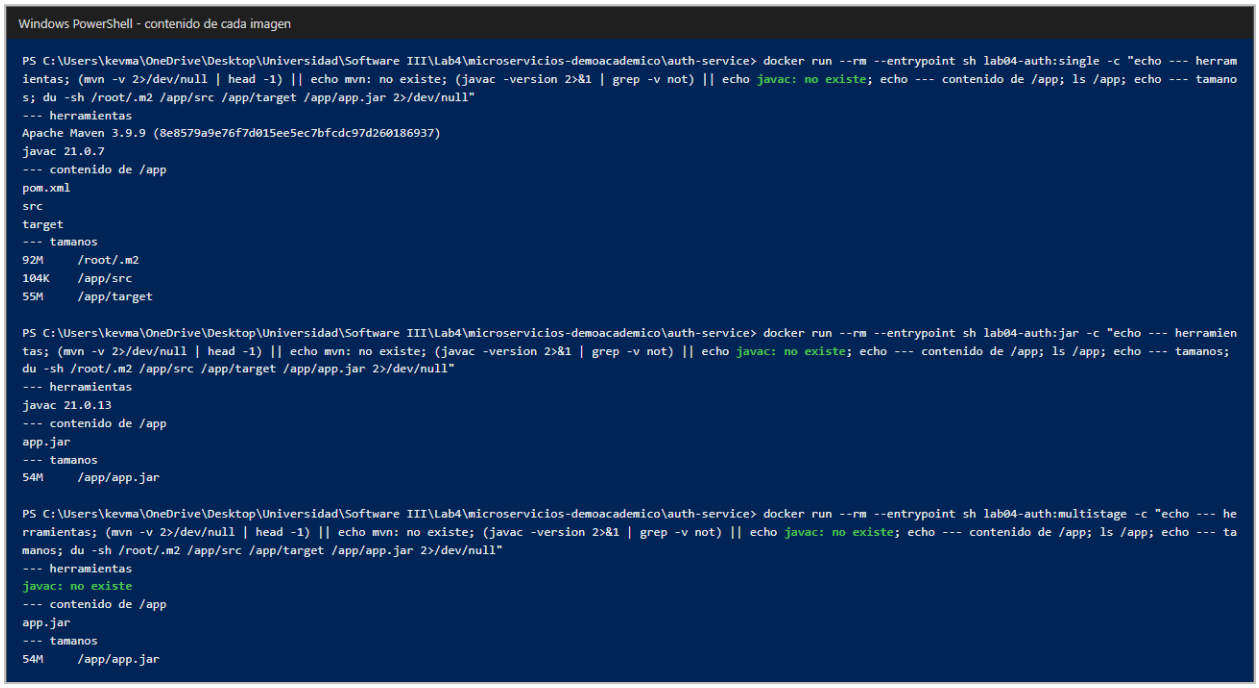
2. ¿Qué archivos adicionales podrían quedar en la imagen final? El código fuente (`src`), el `pom.xml`, la carpeta `target` con clases compiladas y artefactos intermedios, y la caché de dependencias de Maven en `/root/.m2`, que en la prueba ocupó 92 MB.

3. ¿Qué efecto tiene esto sobre el tamaño? La imagen creció hasta 1.07 GB y 14 capas, frente a 564 MB de la versión multi-stage (Figura 23). Una imagen más pesada tarda más en subirse al registro, descargarse en cada servidor y ocupa más disco.

4. ¿Por qué este enfoque no es ideal para producción? Porque mezcla herramientas de construcción con el entorno de ejecución. Aumenta el tamaño y la superficie de ataque: un atacante dentro del contenedor tendría compilador, Maven y el código fuente disponibles. Además, cada cambio de código obliga a volver a resolver dependencias dentro del build.

Figura 22

Herramientas y archivos presentes en cada imagen



Nota. La imagen `single` contiene Maven, `javac`, el código fuente y la caché `.m2`. La imagen `jar` conserva `javac` porque usa la base JDK. La imagen `multistage` solo contiene el JRE y `app.jar`.

Actividad 4: compilación separada

1. ¿Cuál imagen es más grande? La que compila dentro del Dockerfile: 1.07 GB contra 831 MB de la compilación separada (Figura 23).

2. ¿Qué ventaja tiene generar el .jar antes y luego copiarlo? La imagen solo recibe el artefacto final, sin código fuente, Maven ni caché de dependencias. El build de la imagen también es mucho más rápido: unos 6.5 s de Docker, más 6.5 s de Maven local que ya tenía las dependencias en caché.

3. ¿Qué significa, en este caso, separar build y runtime? Que la compilación (`mvn clean package`) ocurre fuera de la imagen, en el equipo del desarrollador o en el servidor de integración continua, y que la imagen solo define cómo ejecutar el JAR (`java -jar app.jar`).

4. ¿Por qué esta separación mejora el despliegue? Porque se despliega una imagen más liviana y con un artefacto ya probado, lo que reduce el tiempo de transferencia y los puntos de falla. Su limitación es que el resultado depende de la máquina que compila, y en este laboratorio la base `eclipse-temurin:21-jdk` todavía incluye el JDK completo, que no hace falta para ejecutar.

Actividad 5: construcción multi-stage

1. ¿Qué ocurre en la etapa build? Sobre la imagen `maven:3.9.9-eclipse-temurin-21` se copia el `pom.xml`, se descargan las dependencias (`dependency:go-offline`, que queda en una capa reutilizable), se copia `src` y se ejecuta `mvn clean package`, que produce el JAR.

2. ¿Qué ocurre en la etapa runtime? Se parte de una imagen nueva y mínima, `eclipse-temurin:21-jre`, se define el directorio de trabajo, se expone el puerto 8081 y se configura el arranque con `java -jar app.jar`.

3. ¿Qué se copia exactamente de una etapa a la otra? Solo el archivo `/build/target/auth-service-0.0.1-SNAPSHOT.jar`, mediante `COPY --from=build`, que queda como `/app/app.jar`.

4. ¿Qué elementos no quedan en la imagen final? Maven, el JDK y `javac`, el código fuente, el `pom.xml`, la caché `.m2`, las clases intermedias de `target` y todas las capas de la etapa build (Figura 22).

5. ¿Qué ventaja tiene multi-stage frente a compilar y ejecutar en una sola etapa? Obtiene una imagen reproducible, porque compila siempre en el mismo entorno, y además

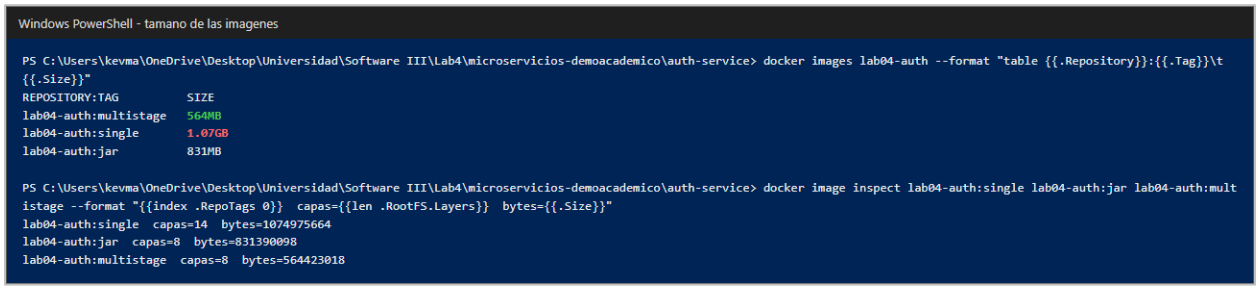
mínima: 564 MB contra 1.07 GB, una reducción de casi 47 %. Todo queda definido en un único Dockerfile, sin depender de que alguien compile antes en su máquina.

Comparación de enfoques y Actividad 6

La Tabla 3 completa la comparación solicitada en la guía con los valores medidos. Los tiempos de build se tomaron sin caché de Docker; el tiempo de despliegue corresponde al arranque del contenedor hasta que `/v3/api-docs` responde.

Figura 23

Tamaño y número de capas de las tres imágenes



Nota. Salida de `docker images` y `docker image inspect`.

Tabla 3

Comparación de los tres enfoques de construcción de la imagen de auth-service

Criterio	Compila dentro del Dockerfile	Compilación separada	Multi-stage
Tamaño de imagen	1.07 GB, 14 capas	831 MB, 8 capas (base JDK)	564 MB, 8 capas (base JRE)
Tiempo de build	79.2 s sin caché	13 s (6.5 s Maven local + 6.5 s Docker)	136.6 s sin caché; luego reutiliza la capa de dependencias
Tiempo de despliegue	Arranque 8.1 s; la más lenta de transferir	Arranque 7.6 s	Arranque 7.8 s; la más rápida de transferir
Seguridad	Baja: Maven, JDK, código y .m2 en la imagen	Media: sin código, pero con JDK y javac	Alta: solo JRE y JAR
Adecuación para producción	No recomendada	Aceptable si el JAR se genera en CI	Recomendada

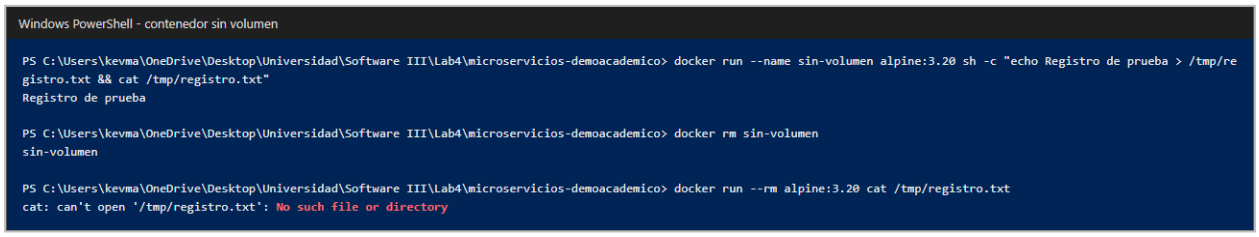
Actividad 6. La tabla muestra que el arranque es casi igual en los tres casos (entre 7.6 y 8.1 s), porque todos ejecutan el mismo JAR. La diferencia real está en el tamaño, que determina cuánto tarda la imagen en subirse al registro y descargarse en cada servidor, y en lo que queda expuesto dentro de ella. La compilación separada construye la imagen más rápido, pero depende de compilar antes fuera de Docker y conserva el JDK. La multi-stage es la más lenta la primera vez, porque descarga las dependencias dentro de Docker, pero produce la imagen más pequeña y segura y es reproducible en cualquier máquina. Por eso es el enfoque más adecuado para producción.

Volúmenes y Actividad 7

Se ejecutaron dos pruebas con la imagen `alpine`. Sin volumen, el archivo escrito en el contenedor desapareció al eliminarlo y un contenedor nuevo no lo encontró (Figura 24). Con el volumen `datos-lab`, el archivo escrito por el contenedor `escritor` siguió disponible para otro contenedor, aunque el primero ya no existía (Figura 25).

Figura 24

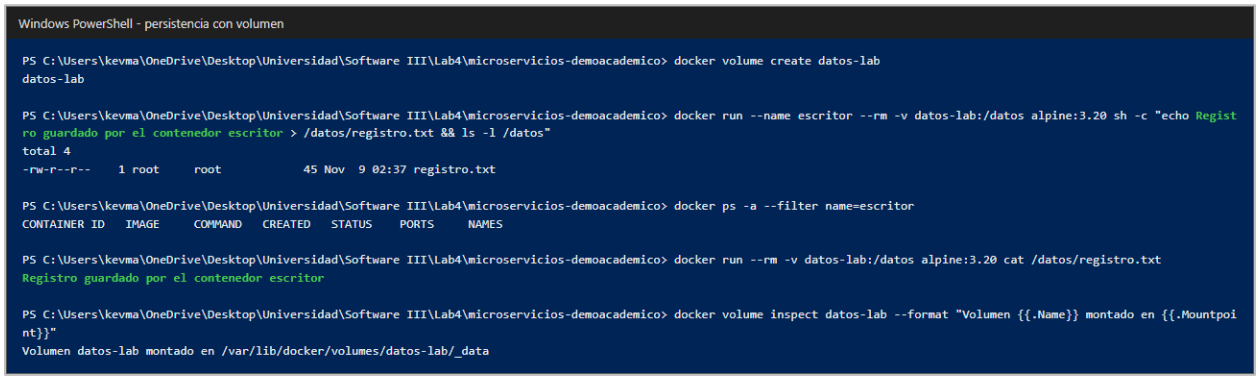
Pérdida de datos al eliminar un contenedor sin volumen



Nota. El segundo contenedor no encuentra `/tmp/registro.txt`.

Figura 25

Persistencia de un archivo con el volumen datos-lab



Nota. El contenedor `escritor` ya no existe, pero el archivo sigue en el volumen.

1. ¿Qué problema resuelven los volúmenes? Separan los datos del ciclo de vida del contenedor. El sistema de archivos de un contenedor es efímero; un volumen lo administra Docker fuera del contenedor, así que los datos sobreviven a su eliminación, recreación o actualización de imagen y pueden compartirse entre contenedores.

2. ¿Qué ocurriría si una base de datos en Docker no usa volumen? Toda la información se perdería al eliminar o recrear el contenedor, por ejemplo al actualizar la imagen o al ejecutar `docker compose down`. Es lo que pasa hoy en este laboratorio con H2 en memoria: al reiniciar, los usuarios y estudiantes se regeneran con los seeders.

3. ¿Por qué este concepto es importante en aplicaciones empresariales? Porque los datos de negocio (clientes, transacciones, matrículas) deben ser durables y respaldables aunque los contenedores sean desechables. Los volúmenes permiten escalar, actualizar o reemplazar contenedores sin perder información y facilitan las copias de seguridad.

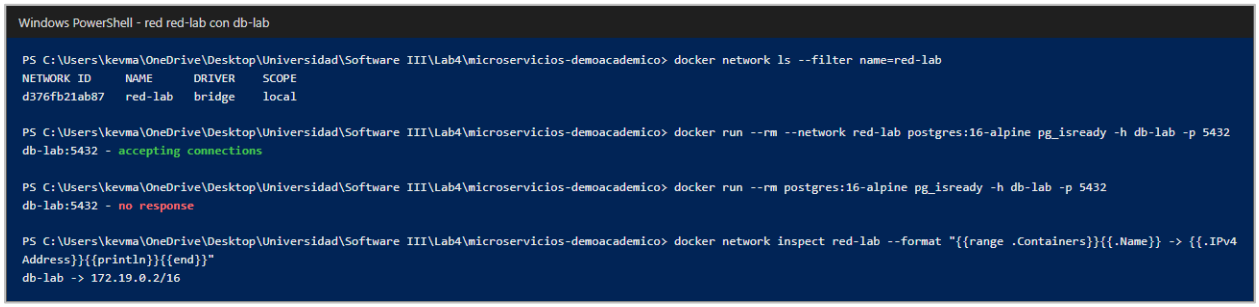
Redes y Actividad 8

Se creó la red `red-lab` con un contenedor PostgreSQL llamado `db-lab`. Un cliente conectado a esa red lo encontró por su nombre y obtuvo "accepting connections"; un cliente

fuera de la red no obtuvo respuesta (Figura 26). En la solución del laboratorio ocurre lo mismo: *consulta-service* resuelve el nombre `auth-service` dentro de `micro-net` (Figura 27).

Figura 26

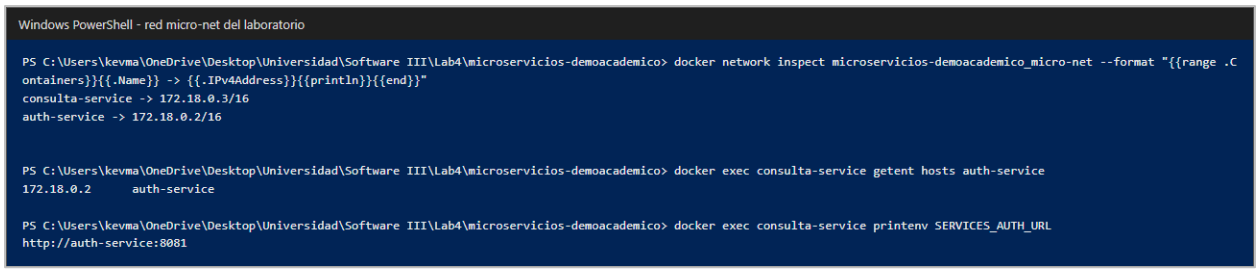
Comunicación por nombre con db-lab dentro y fuera de la red



Nota. `pg_isready` responde "accepting connections" solo desde la red `red-lab`.

Figura 27

Red micro-net de la solución integrada



Nota. `consulta-service` resuelve `auth-service` a 172.18.0.2 y usa `SERVICES_AUTH_URL=http://auth-service:8081`.

1. ¿Para qué sirven las redes en Docker? Para conectar contenedores entre sí de forma aislada. Una red definida por el usuario les da conectividad y un DNS interno para encontrarse por nombre, y deja fuera a los contenedores que no pertenecen a ella.

2. ¿Por qué una aplicación Spring Boot que usa base de datos necesita este concepto? Porque la aplicación y la base de datos corren en contenedores distintos y la aplicación debe alcanzarla desde su URL de conexión (`spring.datasource.url`). Dentro de un

contenedor, `localhost` se refiere al propio contenedor, no a la base de datos, así que ambos necesitan una red común.

3. ¿Qué ventaja tiene poder referirse al contenedor por nombre, por ejemplo db-lab? Las IP de los contenedores cambian cada vez que se recrean (en la prueba, 172.19.0.2 y 172.18.0.2), pero el nombre se mantiene. La configuración queda estable y portable, por ejemplo `jdbc:postgresql://db-lab:5432/academico` o `http://auth-service:8081`, sin editar IP en cada despliegue.

Caso integrador: Actividad 9

La empresa del caso tiene despliegues lentos, imágenes pesadas, pérdida de información al eliminar contenedores y servicios que no se comunican.

1. ¿Qué errores identifica en la construcción de sus imágenes? Compilar dentro de la imagen final, lo que deja Maven, el JDK, el código y la caché `.m2`; usar imágenes base JDK para ejecutar; no tener `.dockerignore`, por lo que se copian `target`, `.git` o archivos del IDE; y ordenar mal las capas, de modo que cada cambio de código vuelve a descargar las dependencias. En la propia solución de este laboratorio se identificó un caso parecido: el Dockerfile de la guía usa `eclipse-temurin:21-jdk` para ejecutar, lo que deja `javac` dentro de la imagen.

2. ¿Qué cambio recomendaría respecto a build y runtime? Separarlos: compilar en una etapa con Maven y JDK y ejecutar en otra con solo el JRE, copiando únicamente el JAR. Además, ejecutar la aplicación con un usuario sin privilegios y agregar un `healthcheck`.

3. ¿Usaría compilación separada o multi-stage? Justifique. Multi-stage. Según la Tabla 3 produce la imagen más pequeña (564 MB) y segura, y no depende de que alguien compile antes en su máquina: el mismo Dockerfile genera el mismo resultado en el equipo del

desarrollador y en el servidor de CI. Su mayor tiempo de primer build se compensa con la caché de la capa de dependencias.

4. ¿Qué papel cumplen los volúmenes en la solución? Garantizan que los datos persistan aunque se eliminen los contenedores, lo que resuelve la pérdida de información del caso. En este laboratorio H2 está en memoria; para hacerlo persistente se configuraría en modo archivo (`jdbc:h2:file:/data/authdb`) o se usaría una base externa, montando un volumen nombrado en esa ruta desde `docker-compose.yml`.

5. ¿Qué papel cumplen las redes en la solución? Permiten que los servicios se encuentren por nombre y se comuniquen de forma aislada. En el laboratorio, la red `micro-net` permite que *consulta-service* valide el token llamando a `http://auth-service:8081`. Los fallos de conexión del caso suelen deberse a usar `localhost` o a dejar los contenedores en redes distintas.

6. ¿Qué mejoraría en tamaño, seguridad, persistencia y despliegue? En tamaño: imágenes multi-stage con base JRE y `.dockerignore`. En seguridad: usuario no root, ningún secreto en la imagen (por ejemplo, pasar `jwt.secret` como variable de entorno o *secret*) y versiones de imagen fijas. En persistencia: volúmenes nombrados para cada base de datos y respaldos periódicos. En despliegue: Docker Compose con `healthcheck` y `depends_on` condicionado a que el servicio esté sano, y publicación de imágenes versionadas en un registro desde un pipeline de CI.

Preguntas guía de reflexión

1. ¿Por qué no compilar en Dockerfile? Porque, si se compila en la etapa final, la imagen de ejecución arrastra herramientas y archivos que no necesita para correr. Compilar

dentro de Docker sí es válido cuando se hace en una etapa separada que luego se descarta, como en multi-stage.

2. ¿Qué sucede con el tamaño de una imagen si incluimos la compilación dentro del Dockerfile? Aumenta de forma considerable: en la prueba pasó de 564 MB (multi-stage) a 1.07 GB, porque se suman Maven, el JDK, la caché de dependencias y los archivos de compilación.

3. ¿Qué ventajas tiene separar la compilación del proceso de ejecución cuando trabajamos con contenedores? Imágenes más pequeñas y rápidas de distribuir, menos componentes expuestos, capas mejor reutilizadas y una responsabilidad clara: la imagen final solo sabe ejecutar el artefacto.

4. ¿Cómo impacta esta separación en la seguridad y en el despliegue en entornos productivos? En seguridad, reduce la superficie de ataque porque no hay compilador, gestor de dependencias ni código fuente que aprovechar, y hay menos paquetes con vulnerabilidades por parchear. En despliegue, una imagen más liviana se descarga y escala más rápido y permite promover el mismo artefacto probado entre ambientes.

5. ¿Qué problema resuelven los volúmenes en Docker? La pérdida de datos por la naturaleza efímera de los contenedores: guardan la información fuera del contenedor para que sobreviva a su eliminación o actualización y pueda compartirse.

6. ¿Qué función cumplen las redes cuando varios contenedores deben comunicarse? Les dan un canal aislado y un DNS interno para encontrarse por nombre de servicio, como `auth-service` o `db-lab`, sin depender de IP cambiantes y sin exponer puertos innecesarios al exterior.

Problemas encontrados y solución

El primer obstáculo fue de entorno: los comandos `java`, `mvn` y `docker` no estaban disponibles en la terminal. Java 21 y Maven ya existían en la instalación de IntelliJ IDEA, por lo

que se utilizaron directamente desde esa ruta. Docker Desktop, en cambio, no se pudo instalar al principio porque requería la característica de Windows *Virtual Machine Platform*. Al ejecutar `wsl --install` como administrador apareció el error `Wsl/CallMsi/Install/REGDB_E_CLASSNOTREG`, que indicaba una instalación de WSL dañada. Se reparó aceptando la reparación automática, que actualizó WSL a la versión 3.0.3; luego se habilitaron con DISM las características `VirtualMachinePlatform` y `Microsoft-Windows-Subsystem-Linux` y, tras reiniciar, Docker Desktop quedó operativo con el motor WSL 2.

En cuanto al código, el repositorio base difiere del ejemplo de la guía: la entidad de estudiante solo tiene nombre y correo, y los paquetes están organizados por funcionalidad. Se decidió conservar el repositorio tal como está y agregar los paquetes nuevos sin reestructurarlo, lo que respeta la arquitectura original y cumple la guía. También se verificó que el puerto de `services.auth.url`, los puertos de cada `application.yml` y el `docker-compose.yml` quedaran alineados; esta coincidencia es necesaria para que la validación funcione tanto localmente como en Docker.

Conclusiones

La separación por capas del proyecto base permitió convertirlo en *consulta-service* con cambios acotados: la lógica de estudiantes no se modificó y la seguridad se añadió como una preocupación transversal mediante un filtro. Separar la autenticación en *auth-service* concentra en un único punto el manejo de credenciales; además, el uso de BCrypt garantiza que una filtración de la base de datos no exponga las contraseñas.

El flujo con JWT evita reenviar credenciales en cada petición, y la validación remota con OpenFeign permite que *consulta-service* no conozca la clave de firma. Como contrapartida, cada petición protegida depende de la disponibilidad de *auth-service*, algo a tener en cuenta en

escenarios de mayor carga. Swagger con el esquema *Bearer* simplificó las pruebas y redujo los errores de formato del encabezado, y Docker Compose permitió levantar la solución completa de forma repetible. Las seis pruebas del guion mínimo produjeron los resultados esperados.

La comparación de imágenes mostró que la forma de construir importa tanto como el código: la versión multi-stage redujo la imagen de 1.07 GB a 564 MB y eliminó del entorno de ejecución el compilador, Maven y el código fuente. Las pruebas de volúmenes y redes confirmaron que los datos deben vivir fuera del contenedor y que los servicios deben encontrarse por nombre dentro de una red común. Como mejoras se proponen adoptar multi-stage con base JRE para ambos servicios y montar volúmenes para que las bases de datos sean persistentes.

Referencias

Docker. (s.f.). *Docker Compose overview*. Docker Docs. https://docs.docker.com/compose/

Fowler, M. y Lewis, J. (2014, 25 de marzo). *Microservices*. martinfowler.com. https://martinfowler.com/articles/microservices.html

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

ojugama. (s.f.). *demo-academico-application* [Código fuente]. GitHub. https://github.com/ojugama/demo-academico-application

OpenAPI Initiative. (2020). *OpenAPI Specification v3.0.3*. https://spec.openapis.org/oas/v3.0.3

Provos, N. y Mazières, D. (1999). A future-adaptable password scheme. En *Proceedings of the 1999 USENIX Annual Technical Conference* (pp. 81-91). USENIX Association.

Spring. (s.f.-a). *Spring Security reference*. https://docs.spring.io/spring-security/reference/

Spring. (s.f.-b). *Spring Cloud OpenFeign*. https://docs.spring.io/spring-cloud-openfeign/reference/